



Е. М. Герасименко

**ИНТЕЛЛЕКТУАЛЬНЫЙ АНАЛИЗ ДАННЫХ.
АЛГОРИТМЫ DATA MINING**

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное
учреждение высшего образования
«Южный федеральный университет»
Инженерно-технологическая академия**

Е. М. Герасименко

**ИНТЕЛЛЕКТУАЛЬНЫЙ АНАЛИЗ ДАННЫХ.
АЛГОРИТМЫ DATA MINING**

Учебное пособие

**Ростов-на-Дону – Таганрог
Издательство Южного федерального университета**

2017

УДК 004.8(075.8)
ББК 32.973я73
Г37

Рецензенты:

академик РАН, заслуженный деятель науки РФ, доктор технических наук,
профессор, зав. кафедрой ДМиМО ИКТИБ Южного федерального университета
Курейчик В.М.;

кандидат физико-математических наук, доцент, зав. кафедрой математики
Таганрогского института имени А.П. Чехова (филиала) «Ростовского
государственного экономического университета (РИНХ)», *Ляхова Н.Е.*

Герасименко, Е. М.

Г37 Интеллектуальный анализ данных. Алгоритмы Data Mining : учебное пособие /
Е. М. Герасименко; Южный федеральный университет. – Ростов-на-Дону –
Таганрог : Издательство Южного федерального университета, 2017. – 84 с.
ISBN 978-5-9275-2388-7

В данном учебном пособии рассматриваются теоретические положения и методические указания к выполнению лабораторных работ по курсу «Интеллектуальный анализ данных, биоинспирированные методы и алгоритмы» Пособие включает 4 лабораторные работы по кластеризации, анализу текста, классификации, коллаборативной фильтрации в среде «Orange», а также методические указания по их выполнению. Освещаются основные алгоритмы кластеризации, вводится понятие очистки данных. Излагаются некоторые подходы к анализу текста, особое внимание уделяется тональному анализу текста. Приводятся различные алгоритмы классификации, изучаются способы их сравнения и оценки качества. Рассматриваются подходы к организации рекомендательных систем, некоторые алгоритмы коллаборативной фильтрации.

Пособие предназначено для магистрантов высших учебных заведений, обучающихся по направлению 09.04.01 «Информатика и вычислительная техника», изучающих дисциплину «Интеллектуальный анализ данных, биоинспирированные методы и алгоритмы». Также оно может быть интересно студентам, аспирантам, изучающим дисциплину «Интеллектуальный анализ данных», и специалистам в данной области.

ISBN 978-5-9275-2388-7

УДК 004.8(075.8)
ББК 32.973я73

© Южный федеральный университет, 2017
© Герасименко Е. М., 2017

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1. КЛАСТЕРИЗАЦИЯ	8
1.1. Понятие кластеризации	8
1.2. Классификация методов кластеризации	9
1.2.1. <i>Иерархическая кластеризация</i>	10
1.2.2. <i>Алгоритмы квадратичной ошибки (k-средних)</i>	10
1.2.3. <i>Нечеткие алгоритмы кластеризации (c-средних)</i>	11
1.2.4. <i>Алгоритм максимизации ожидания (Expectation-Maximization)</i>	11
1.2.5. <i>Графовые методы</i>	12
1.2.5.1. <i>Алгоритм выделения связных компонент</i>	12
1.2.5.2. <i>Алгоритм минимального покрывающего дерева</i>	13
1.2.5.3. <i>Послойная кластеризация</i>	13
1.3. Очистка данных	14
Лабораторная работа №1. КЛАСТЕРИЗАЦИЯ. ПОСТРОЕНИЕ МОДЕЛИ КЛАСТЕРИЗАЦИИ В ORANGE	15
Контрольные вопросы и задания	24
2. WEB MINING И TEXT MINING	25
2.1. Web mining	25
2.2. Интеллектуальный анализ текста. Тональный анализ	26
2.2.1. <i>Анализ на основе правил</i>	27
2.2.2. <i>Анализ на основе словаря</i>	27
2.2.3. <i>Машинное обучение с учителем</i>	28
2.2.4. <i>Машинное обучение без учителя</i>	28
2.3. Проблемы тонального анализа	28

Лабораторная работа №2. ИНТЕЛЛЕКТУАЛЬНЫЙ АНАЛИЗ ТЕКСТА. WEB MINING. ТОНАЛЬНЫЙ АНАЛИЗ ТЕКСТА В ORANGE	29
Контрольные вопросы и задания	39
3. КЛАССИФИКАЦИЯ	40
3.1. Понятие классификации.....	40
3.2. Ирисы Фишера	41
3.3. Процесс классификации	43
3.4. Алгоритмы классификации	45
3.4.1. <i>Дерево решений</i>	45
3.4.2. <i>Наивный байесовский классификатор</i>	47
3.4.3. <i>Метод опорных векторов (Support Vector Machines)</i>	48
3.4.4. <i>Метрические классификаторы</i>	49
3.4.4.1. <i>Метод ближайшего соседа (nearest neighbor)</i>	50
3.4.4.2. <i>Метод k ближайших соседей (k nearest neighbors)</i>	51
3.4.5. <i>Случайный лес</i>	51
3.5. Точность классификации: оценка уровня ошибок	52
3.6. Численная оценка качества алгоритма	54
3.6.1. <i>Точность (Accuracy)</i>	54
3.6.2. <i>Точность и полнота</i>	55
3.6.3. <i>Confusion Matrix</i>	56
3.6.4. <i>F-мера</i>	57
3.6.5. <i>ROC-кривая</i>	58
Лабораторная работа №3. КЛАССИФИКАЦИЯ. ПОСТРОЕНИЕ МОДЕЛИ КЛАССИФИКАЦИИ. СРАВНЕНИЕ РАЗЛИЧНЫХ АЛГОРИТМОВ КЛАССИФИКАЦИИ И ВЫБОР ОПТИМАЛЬНОГО В ORANGE.....	61
Контрольные вопросы и задания	64

4. РЕКОМЕНДАЦИИ. КОЛЛАБОРАТИВНАЯ ФИЛЬТРАЦИЯ	65
4.1. Методы построения рекомендательных систем	66
4.2. Алгоритм Slope One.....	67
4.3. Проблемы методов коллаборативной фильтрации	70
4.4. Сингулярное разложение матрицы (Matrix Factorization with SVD).....	72
4.5. Оценка качества работы рекомендательных систем.....	73
Лабораторная работа №4. РЕКОМЕНДАТЕЛЬНЫЕ СИСТЕМЫ. АЛГОРИТМЫ КОЛЛАБОРАТИВНОЙ ФИЛЬТРАЦИИ. АЛГОРИТМ SLOPE ONE. СРАВНЕНИЕ РЕЗУЛЬТАТОВ РАБОТЫ АЛГОРИТМОВ КОЛЛАБОРАТИВНОЙ ФИЛЬТРАЦИИ В ORANGE	74
Контрольные вопросы и задания	76
ЗАКЛЮЧЕНИЕ	78
БИБЛИОГРАФИЧЕСКИЙ СПИСОК.....	80

ВВЕДЕНИЕ

Целями изучения дисциплины «Интеллектуальный анализ данных, биоинспирированные методы и алгоритмы» являются:

- изучение современных подходов к интеллектуальному анализу данных;
- изучение теории эволюционных вычислений, а также создания и практического использования методов и алгоритмов, инспирированных природными системами;
- формирование навыков самостоятельной постановки задач и обоснование выбора метода их решения с использованием современных алгоритмов интеллектуального анализа данных.

Задачи дисциплины

1. Углубленное изучение методов оптимизации многокритериальных задач и составление математических моделей оптимизационных задач.
2. Формирование навыков создания моделей интеллектуального анализа данных в современных программных средах.
3. Исследование методов извлечения информации из текстовых данных.
4. Изучение основных подходов к классификации изображений.
5. Формирование навыков построения биоинспирированных алгоритмов и применения тех или иных методов и моделей в зависимости от решаемой задачи.

Настоящее учебное пособие включает материалы, которые составляют первую часть лабораторного практикума по курсу «Интеллектуальный анализ данных, биоинспирированные методы и алгоритмы». На лабораторных работах магистранты изучают основные методы интеллектуального анализа данных, применяют их на практике с помощью современных программных сред.

Пособие освещает 4 лабораторные работы по кластеризации, анализу текста, классификации и коллаборативной фильтрации и содержит теоретические сведения по курсу «Интеллектуальный анализ данных, биоинспирированные методы и алгоритмы», что позволяет магистрантам глубже понять суть алгоритмов интеллектуального анализа данных, интерпретировать полученные результаты,

ответить на контрольные вопросы и сформулировать заключение.

Учебное пособие опирается на современные учебные пособия, статьи, сборники трудов международных конференций по интеллектуальному анализу данных, в частности, кластеризации, классификации, анализу текста, рекомендательным системам, а также на ресурсы сети Интернет в названных выше областях.

1. КЛАСТЕРИЗАЦИЯ

1.1. Понятие кластеризации

Задача кластеризации заключается в разбиении множества объектов на какие-либо группы таким образом, чтобы внутри этих групп находились максимально близкие, похожие элементы, а элементы, находящиеся в разных группах, максимально отличались. Кластеризация имеет сходство с задачей классификации в аспекте разбиения на классы, но является более сложной задачей, так как классы должны определиться в процессе выполнения процедуры, а не быть заранее известными. В некоторых ситуациях данные, которыми оперируют алгоритмы кластеризации, содержат большое количество записей (их количество может достигать миллионов), а записи, в свою очередь, содержат множество признаков (атрибутов) различных типов [1].

Множество признаков, или атрибутов, можно разделить на числовые, описываемые числом, и нечисловые, или «категорийные», означающие абстрактное понятие. В частности, атрибуты «широта» и «долгота» относятся к числовым, а «образование» или «цвет» – к категорийным.

В основе методов кластеризации лежит сравнение объектов и определение меры их сходства. Мера сходства (близости) – величина, имеющая предел и отражающая сходство объектов так, что с увеличением их сходства она возрастает. Существуют различные меры близости объектов, их выбор обусловлен спецификой решаемой задачи и шкалой измерений (Евклидово, Манхэттенское расстояние, мера Чебышева и пр.). Пример оценки сходства числовых атрибутов может быть представлен евклидовым расстоянием, вычисляемым по формуле

$$D(x, y) = \sqrt{\sum_i (x - y)^2}. \quad (1.1)$$

В (1.1) x и y – координаты точки на плоскости.

Для оценки сходства нечисловых атрибутов используют меры Жаккара, Танимото, Кульчинского и пр. Например, коэффициент Жаккара отражает

отношение числа уникальных символов в двух множествах к общему числу уникальных символов.

1.2. Классификация методов кластеризации

Существует множество классификаций алгоритмов кластеризации. Приведем наиболее распространенную классификацию, согласно которой алгоритмы кластеризации подразделяются:

- на иерархические и плоские. В основе иерархических алгоритмов кластеризации – построение вложенных разбиений, таким образом, меньшие кластеры объединяются в большие или, наоборот, большие последовательно делятся на меньшие. В процессе работы алгоритма формируется дерево кластеров. Корнем дерева является вся выборка, а листьями – наиболее мелкие кластеры.

Плоские алгоритмы строят одно общее разбиение объектов на кластеры [2]. К ним относят алгоритмы квадратичной ошибки (k -средних):

- масштабируемые и немасштабируемые. Масштабируемые алгоритмы адаптируются к ограниченным ресурсам вычислительной техники (объем памяти, быстродействие) и обеспечивают линейный рост времени работы с увеличением числа исследуемых записей. Немасштабируемые алгоритмы подразумевают работу со всеми данными сразу, поэтому они крайне чувствительны к объему вычислительных ресурсов;

- четкие и нечеткие. Ключевое отличие четких алгоритмов кластеризации от нечетких связано с понятием степени принадлежности, положенным в основу нечетких множеств. Так, четкие алгоритмы ставят в соответствие объекту только один кластер, в то время как в нечетких алгоритмах объект может принадлежать разным кластерам с разной степенью принадлежности или же принадлежать какому-либо кластеру со степенью принадлежности, меньшей единицы. Следовательно, нечеткие алгоритмы кластеризации содержат сомнение, неопределенность относительно принадлежности элемента кластеру [2].

Рассмотрим алгоритмы кластеризации подробнее.

1.2.1. Иерархическая кластеризация

Алгоритмы иерархической кластеризации подразделяются на *восходящие* и *нисходящие*.

Принцип работы нисходящих алгоритмов подразумевает прохождение «сверху-вниз»: вначале из всех объектов формируется единый кластер, последовательно разделяемый в процессе на более мелкие кластеры [3].

Восходящие алгоритмы получили большее распространение. Принцип их работы заключается в последовательном укрупнении кластеров с дальнейшим соединением кластеров в один крупный кластер, который будет содержать всю выборку. Данная схема построения создает на каждом шаге вложение разбиений в более крупные. Визуальным представлением данных алгоритмов является дерево – дендрограмма, примером которой является классификация растений [2].

1.2.2. Алгоритмы квадратичной ошибки (*k*-средних)

Задача кластеризации, как было упомянуто выше, состоит в разбиении множества на классы оптимальным образом. Следовательно, нам необходимо определить критерий оптимальности этого разбиения. В качестве критерия оптимальности будем использовать показатель минимизации среднеквадратической ошибки разбиения, определяемый формулой

$$e^2(X, L) = \sum_{j=1}^K \sum_{i=1}^{n_j} \|x_i^{(j)} - c_j\|^2. \quad (1.2)$$

В (1.2) c_j – «центр масс» кластера j (точка со средними значениями характеристик для данного кластера).

Алгоритмы квадратичной ошибки относятся к плоским алгоритмам, наиболее известным из которых является метод *k-средних* (*k-means*). Алгоритм находит заданное число максимально отдаленных друг от друга кластеров [3]. Опишем основные шаги алгоритма:

1. На первом этапе произвольно выбираются k начальных «центров» кластеров.
2. Каждому объекту ставят в соответствие кластер с ближайшим «центром».

3. После изменений необходимо определить (пересчитать) новые «центры» кластеров.

4. Если критерий остановки алгоритма достигнут, – конец, в противном случае – перейти к шагу 2.

Алгоритм может заканчивать работу в следующих случаях: при минимальном колебании среднеквадратической ошибки или если второй этап не дал переход объекта в другой кластер [2].

Ключевым (но решаемым) недостатком алгоритмов *k-средних* является требование к заданию количества кластеров, на которые необходимо разбить множество.

1.2.3. Нечеткие алгоритмы кластеризации (*c-средних*)

Разновидностью метода *k-средних* является его нечеткая разновидность – алгоритм *c-средних* (*c-means*). Представим данный алгоритм.

1. Задать матрицу U , содержащую значения степеней принадлежности, ее размер – $[n, k]$, где n – количество объектов, а k – количество кластеров, и определить исходное разбиение объектов на k кластеров.

2. По матрице определить величину нечеткой ошибки, представленную формулой

$$E^2(X, U) = \sum_{i=1}^N \sum_{k=1}^K U_{ik} \|x_i^{(k)} - c_k\|^2. \quad (1.3)$$

В (1.3) c_k – «центр масс» нечеткого кластера k : $c_k = \sum_{i=1}^N U_{ik} x_i$.

3. Уменьшить нечеткую ошибку, перераспределив элементы.

4. Перейти к п. 2 в случае существенных изменений матрицы U .

К недостаткам метода можно отнести необходимость задания начального количества кластеров [2].

1.2.4. Алгоритм максимизации ожидания (*Expectation-Maximization*)

Алгоритм максимизации ожидания («Expectation-Maximization», или ЕМ-алгоритм) относится к числу неиерархических алгоритмов, поскольку он оперирует

вероятностью, а не расстоянием. Так, ЕМ-алгоритм определяет вероятность попадания конкретного элемента в кластер с помощью анализа исходных данных, а затем параметры модели пересчитываются согласно результатам предыдущего шага. Эти этапы повторяются до тех пор, пока распределение по кластерам и параметры модели не достигнут максимального правдоподобия.

Метод эффективно оперирует большим объемом данных, обладает быстрой сходимостью, сложность его увеличивается линейно с ростом количества данных. Главный недостаток связан с необходимостью задания вида распределения в начале алгоритма, которое трудно оценить в силу сложности анализа большого массива данных.

1.2.5. Графовые методы

Графовые методы кластеризации позволяют представить исходное множество кластеризуемых объектов в виде графа $G=(V, E)$, вершины которого представляют элементы множества, а ребра имеют вес, определяемый по расстоянию между объектами. Традиционно графовые представления характеризуются наглядностью, простотой модификаций. Далее рассмотрены некоторые графовые алгоритмы кластеризации.

1.2.5.1. Алгоритм выделения связных компонент

Данный алгоритм основан на понятии «компонента связности» графа, где последняя трактуется как максимальный связный подграф, т.е. такой подграф, между любой парой вершин которого есть путь. Пусть вершины графа G – это объекты исследуемой выборки, а ребра графа G есть «расстояния» между объектами. Зададим параметр R и удалим из графа все ребра, для которых «веса-расстояния» больше параметра R . Связанными при этом останутся ближайшие друг к другу объекты. Задача алгоритма состоит в подборе такого значения параметра R , которое приведет к образованию нескольких компонент связности – кластеров.

Подбор параметра R основан на построении гистограммы распределений попарных «расстояний-весов». В типах задачах с выраженным разбиением исходной выборки на кластеры гистограмма имеет два пика. Первый характеризует

расстояние между элементами кластера внутри заданного кластера, а второй – расстояние между кластерами. Минимальное значение между данными точками определяет параметр R . К недостаткам метода относят неэффективность его работы для разреженных данных, а также сложность управления количеством кластеров, исходя из параметра R [2].

1.2.5.2. Алгоритм минимального покрывающего дерева

Данный алгоритм представляет собой построение графа в виде так называемого минимального остоного (покрывающего) дерева. Это дерево имеет минимальный суммарный вес входящих в него ребер, где вес представляет собой длину, или расстояние. Затем из построенного дерева удаляем ребра с наибольшим весом. Минимальное покрывающее дерево для 12 элементов (вершин) представлено на рис. 1.1.

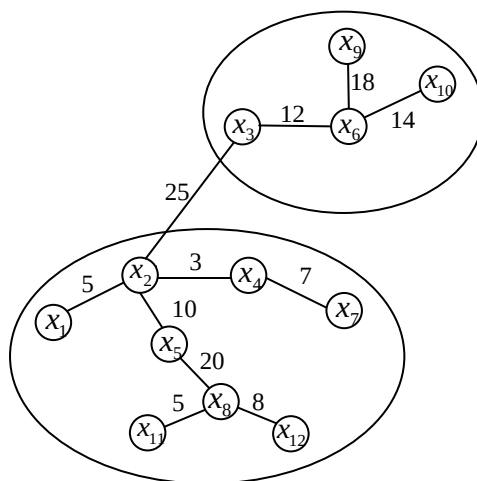


Рис. 1.1. Минимальное покрывающее дерево

Удаляем связь с наибольшим весом, равным 25 единицам, – (x_2, x_3) . В результате исходное дерево разбивается на два кластера: $\{x_1, x_2, x_4, x_5, x_7, x_8, x_{11}, x_{12}\}$ и $\{x_3, x_6, x_9, x_{10}\}$. Первый кластер на следующем шаге также может быть разделен на два кластера путём удаления ребра $\{x_5, x_8\}$ (расстояние равно 20 единицам) и т.д. [2].

1.2.5.3. Послойная кластеризация

Алгоритм послойной кластеризации так же, как и алгоритм выделения связных компонент, заключается в поиске связных компонент. Но его отличие в том, что компоненты связности определяются на определенном уровне весов между

вершинами графа ρ , где уровень соотносится с расстоянием c . Например, если расстояние между объектами $0 \leq \rho(x, x') \leq 1$, то $0 \leq c \leq 1$.

Алгоритм послойной кластеризации формирует последовательность подграфов графа G , которые отражают иерархические связи между кластерами:

$$G^0 \subseteq G^1 \subseteq \dots \subseteq G^m,$$

где $G_t = (V, E_t)$ – граф на уровне c_t ,

$$E^t = \{e_{ij} \in E : \rho_{ij} \leq c_t\},$$

c_t – t -й порог расстояния,

m – количество уровней иерархии,

$G_0 = (V, \emptyset)$, $\emptyset$ – пустое множество ребер графа, получаемое при $t_0 = 1$,

$G_m = G$, т.е. граф объектов без ограничений на расстояние (длину ребер графа), поскольку $t_m = 1$.

Изменяя пороги весов $\{c_0, \dots, c_m\}$, где $0 = c_0 < c_1 < \dots < c_m = 1$, можно управлять глубиной иерархии получаемых кластеров. Таким образом, алгоритм послойной кластеризации может создавать как плоское разбиение данных, так и иерархическое [2].

1.3. Очистка данных

Перед началом обработки данных нужно произвести очистку информации. Из выборки следует убрать те значения и атрибуты, которые не влияют на конечный результат и являются информационным шумом.

Например, при разделении покупателей торговых сетей на группы по товарным предпочтениям, целесообразно исключить из рассмотрения рекламную продукцию, выданные дисконтные карты, тару и пакеты, покупаемые на кассе, в связи с тем, что эти объекты никак не относятся к поведению покупателей и в данной ситуации являются шумом.

Задача очистки данных представляет собой трудноформализуемую задачу и подразумевает аналитический (экспертный подход), так как критерий отнесения тех или иных данных к шуму вырабатывается исключительно в рамках конкретной задачи.

Лабораторная работа №1. КЛАСТЕРИЗАЦИЯ. ПОСТРОЕНИЕ МОДЕЛИ КЛАСТЕРИЗАЦИИ В ORANGE

Цель и задача работы

Изучить основные методы кластеризации с использованием приложения «Orange Data Mining».

Используя различные алгоритмы кластеризации, выявить оптимальные точки размещения отделений неотложной медицинской помощи в регионе на основе выданных тестовых данных.

Ход работы

Orange – свободно распространяемая библиотека, написанная на языке Python, основанная на принципе визуального программирования для наглядного доступа к алгоритмам Data mining. Разработана и поддерживается Bioinformatics Laboratory of the Faculty of Computer and Information Science Люблянского университета (Словения). **Orange** предоставляет пользователю следующие основные функции [4]:

- Загрузка данных из различных источников (файлы, веб-ресурсы, базы данных) и представление их в табличном виде.
- Получение информации об атрибутах данных (полях таблицы).
- Построение потока data mining (data mining workflow).
- Изменение данных и параметров «на лету» (что позволяет отследить изменения в режиме реального времени).
- Визуализация результатов с помощью различных графиков.
- Сохранение модели и применение её в дальнейшем.

Пример построения модели Data mining в Orange

*Загрузка данных (виджет **File**)*

Orange поставляется со своим собственным форматом данных, но также может работать с другими форматами, например, Excel (.xlsx или .xls) или CSV-файлами. Как правило, входными данными является таблица с записями (объектами) в строках и атрибутами данных в столбцах. Атрибуты могут быть разного типа (непрерывные, дискретные и строковые). Типы атрибутов и их вид

могут быть представлены в заголовке таблицы. Они также могут быть впоследствии изменены в виджете **File**. Тип данных также может быть изменен с помощью виджета **Select Columns**.

Виджет **File** находится на вкладке **Data**. Пример простой модели Orange с использованием виджетов **File** и **Data Table** показан на рис. 1.2 (в качестве источника данных выбран файл на локальном компьютере).

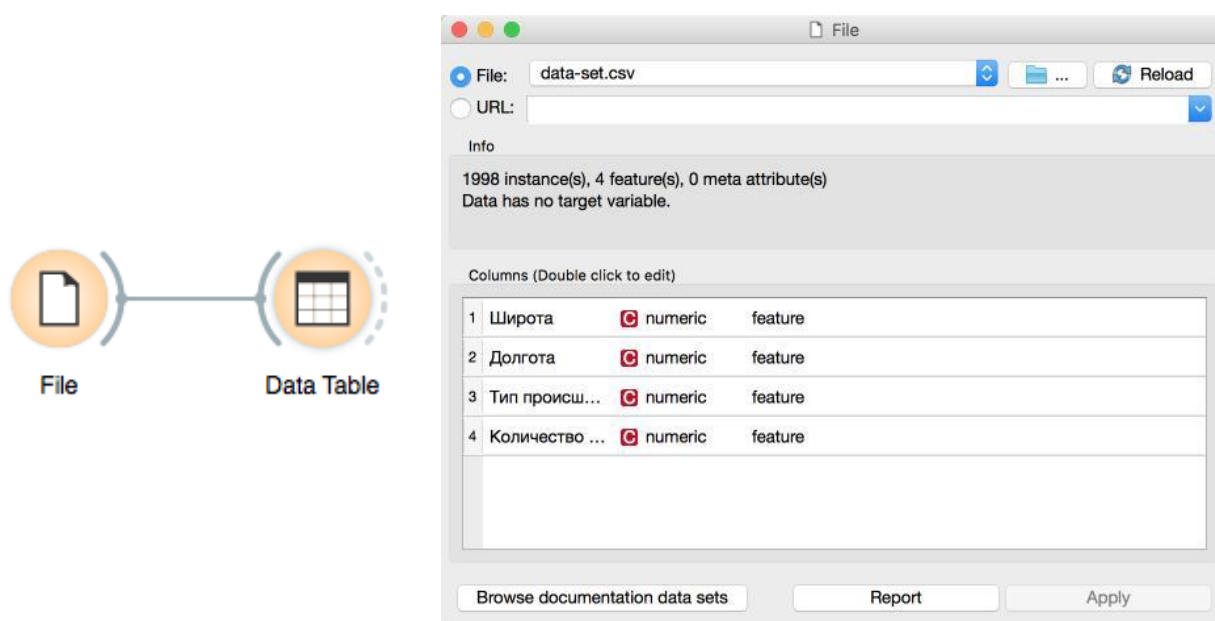


Рис. 1.2. Загрузка файла данных и отображение его содержимого в виде таблицы

Очистка/фильтрация данных (виджеты **Select Rows** и **Select Columns**)

Для простой очистки данных по значению какого-либо атрибута можно использовать виджет **Select Rows** (выбор строк). Данный виджет позволяет задать различные условия для фильтрации входных данных. Например, у нас есть список пользователей и нам нужно исследовать только мужчин в возрасте от 25 лет и старше. Тогда настройки виджета **Select Rows** будут иметь вид, как показано на рис. 1.3.

Допустим, что в приведенном выше примере нам нужно исследовать все поля, кроме поля ID. Для этого мы применим виджет **Select Columns** с настройками, показанными на рис. 1.4.

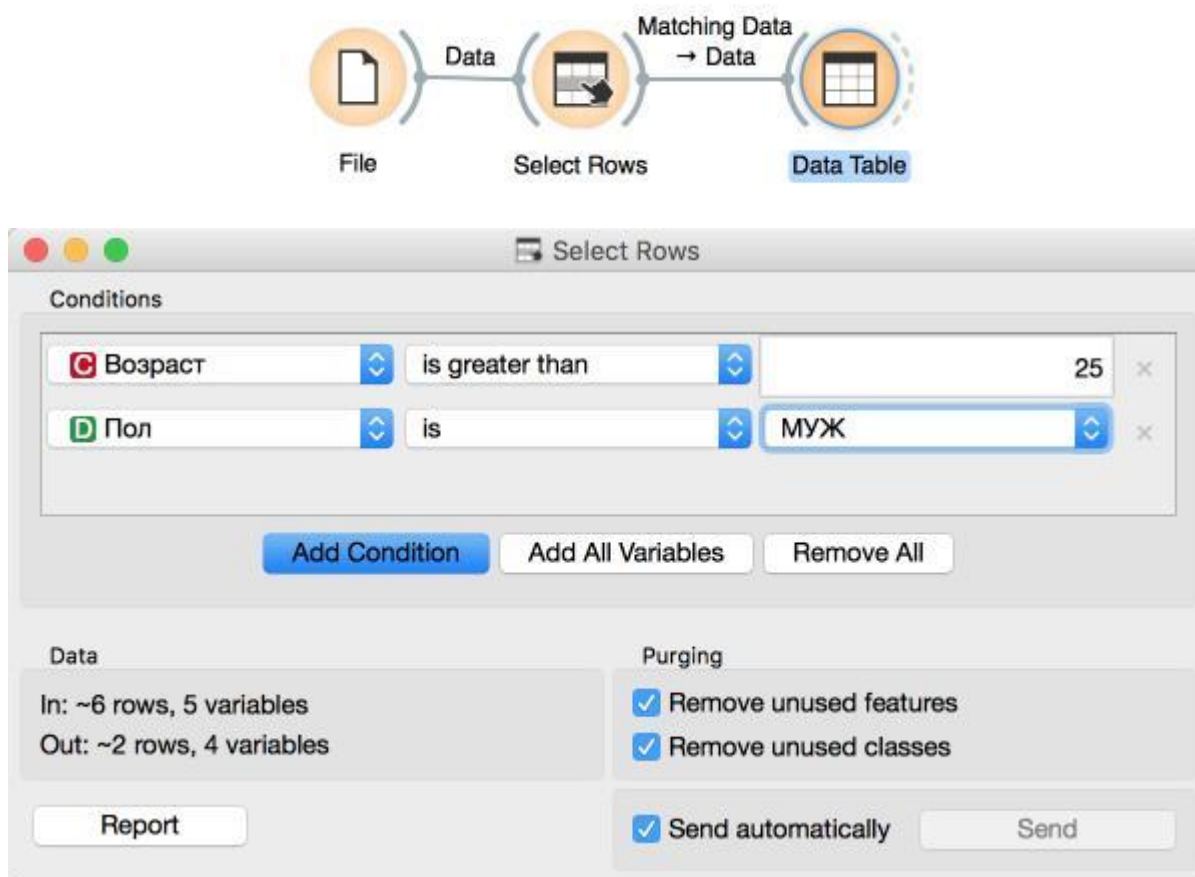


Рис. 1.3. Фильтрация данных с помощью виджета **Select Rows**

Можно самостоятельно придумать некоторый набор данных и посмотреть, как будут меняться выходные данные с помощью виджета **Data Table**. Данный виджет хорошо подходит для отображения табличных данных, полученных на выходе других виджетов.

Все описанные виджеты также находятся на вкладке **Data**.

Использование виджета k-Means

Виджет алгоритма k-Means находится на вкладке **Unsupervised**. После необходимой подготовки данных мы можем применить данный алгоритм. Для этого создадим следующую модель Data mining (рис. 1.5).

Остановимся подробнее на возможных настройках данного виджета. Так, мы можем устанавливать количество кластеров, на которые алгоритм пытается разделить наши данные (группа Number of Clusters на рис. 1.6).

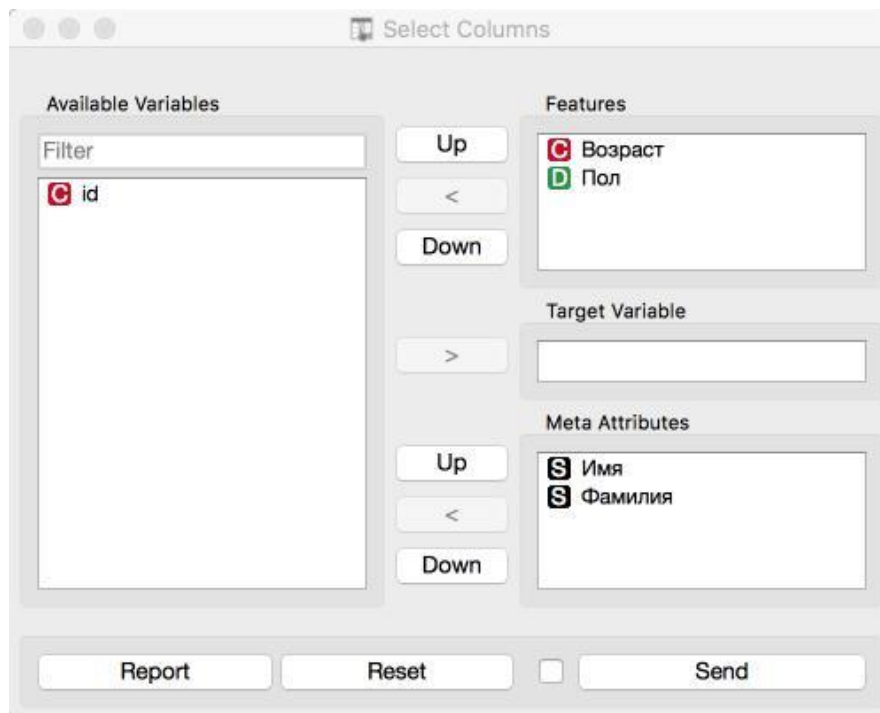
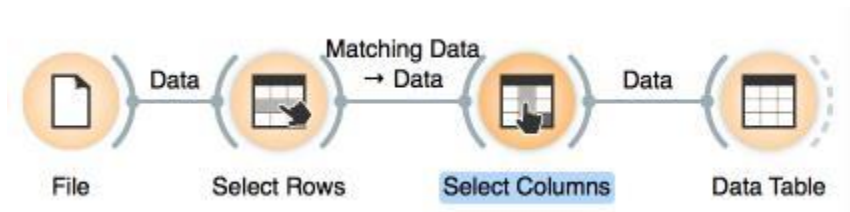


Рис. 1.4. Выбор необходимых столбцов с помощью виджета **Select Columns**

Доступно два варианта: фиксированное (Fixed) количество кластеров и подобранное (Optimized). При выборе подбора количества кластеров необходимо задать минимальное и максимальное количество кластеров.

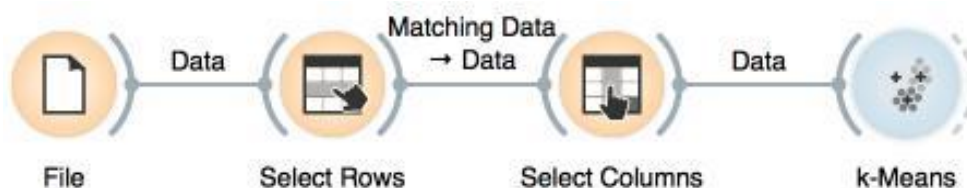


Рис. 1.5. Добавление виджета алгоритма кластеризации k-Means

Также в блоке **Number of Cluster** можно выбрать критерий оценки качества кластеризации (Scoring) (рис. 1.6). Для выбора доступны следующие варианты:

- Silhouette – мера того, как сильно принадлежит объект своему кластеру и как сильно он отличается от объектов других кластеров. Другими словами, как среднее расстояние до объектов своего кластера соотносится со средним расстоянием до объектов других кластеров;

- Inter-cluster distance – мера расстояния между центрами кластеров;
- Distance to centroids – мера расстояния до объектов, соответствующих среднему арифметическому значению кластера.

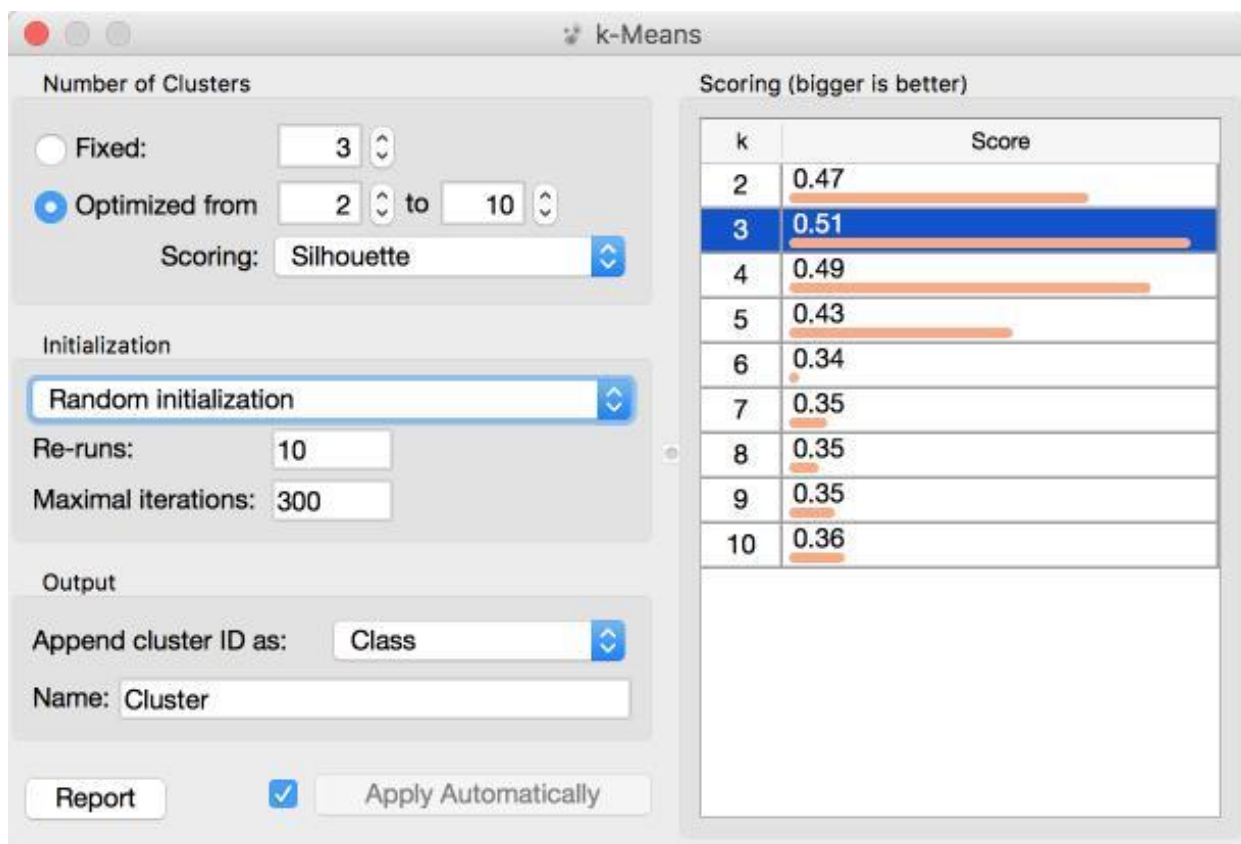


Рис. 1.6. Настройки виджета алгоритма k-Means

В ходе лабораторной работы важно посмотреть, как влияет выбор критерия качества кластеризации на результат работы алгоритма.

Также на рис. 1.6 можно видеть блок **Initialization**. Он позволяет изменить алгоритм начального выбора центров кластеров. Доступны 2 значения:

1. k-Means++ заключается в том, что первые центры кластеров выбираются случайно, последующие центры выбираются из оставшихся объектов с вероятностью, пропорциональной квадрату расстояния до ближайшего центра.

2. Random initialization – центры кластеров выбираются случайно и изменяются по ходу работы алгоритма.

Визуализация с помощью виджета Scatter Plot

Виджет **Scatter Plot** находится на вкладке **Visualize**. Он позволяет строить диаграмму рассеивания (или точечную диаграмму). Добавим к предыдущему примеру модели data mining виджет **Scatter Plot** (рис. 1.7).



Рис. 1.7. Визуализация данных с помощью виджета **Scatter Plot**

В настройках виджета можно выбирать оси координат (Axis x, Axis y), менять размер, цвет и вид маркеров (группа Points). Также имеется возможность сохранять полученную диаграмму в виде файла (кнопка **Save Image**).

Использование виджета Hierarchical Clustering

Виджет алгоритма иерархической кластеризации (**Hierarchical Clustering**) находится на вкладке **Unsupervised**. После необходимой подготовки данных мы

может применить данный алгоритм. Для этого дополним предыдущую модель data mining (рис. 1.8).

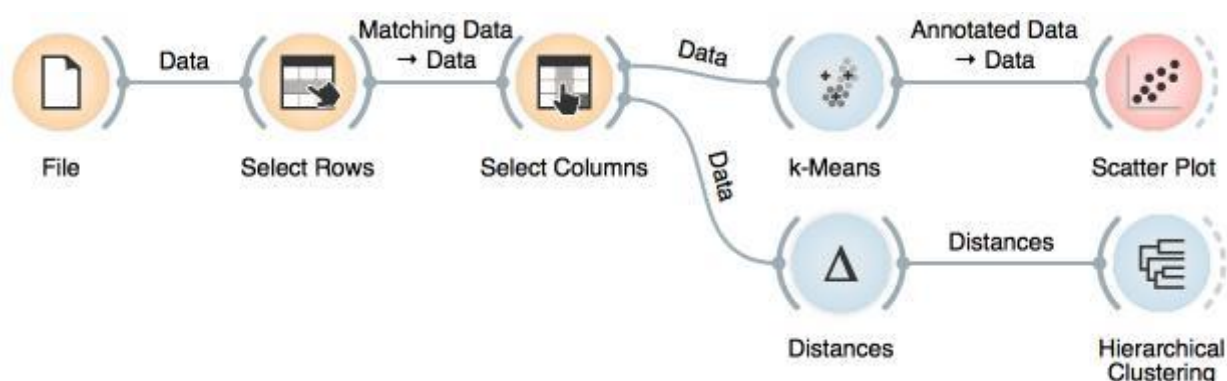


Рис. 1.8. Использование виджета **Hierarchical Clustering** совместно с виджетом **Distances**

Алгоритм иерархической кластеризации не может работать напрямую с входными данными, для работы ему необходима матрица расстояний. Для этого мы и добавляем виджет **Distances**.

Рассмотрим возможные настройки виджета **Distances** (рис. 1.9).

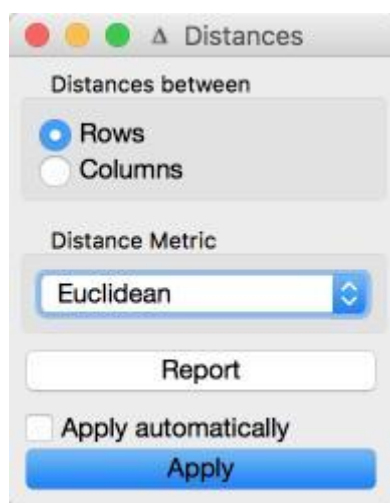


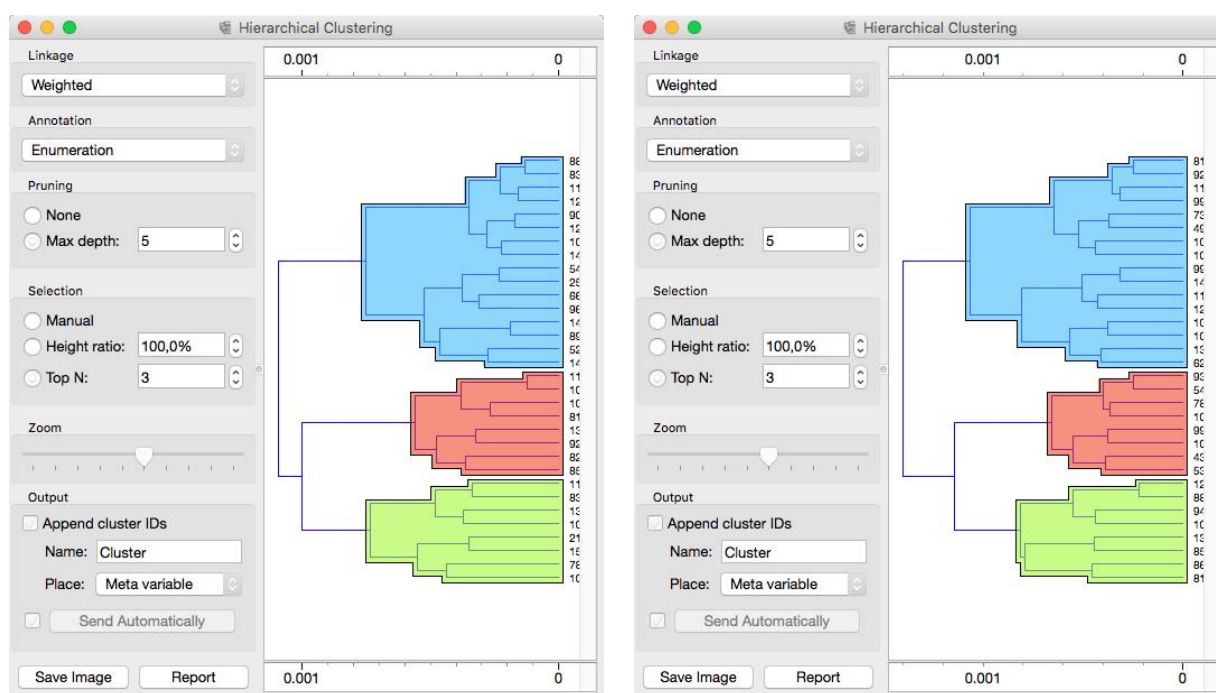
Рис. 1.9. Настройки виджета **Distances**

Наиболее интересной является возможность менять меру расстояния (**Distance Metric**). Так, доступны следующие варианты:

- Euclidean – эвклидово расстояние;
- Manhattan – также известно как расстояние городских кварталов: расстояние между двумя точками, равное сумме модулей разностей их координат;

- Cosine – косинусный коэффициент, или коэффициент сходства, определяется как косинус угла между двумя векторами внутреннего пространства объектов;
- Jaccard – мера Жаккара: размер пересечения делится на размер объединения множеств выборок;
- и другие.

В качестве эксперимента попробуйте выбрать некоторые из приведенных мер расстояний и отследите характер изменения результатов кластеризации. Так, диаграммы для Euclidian и Manhattan приведены на рис. 1.10.



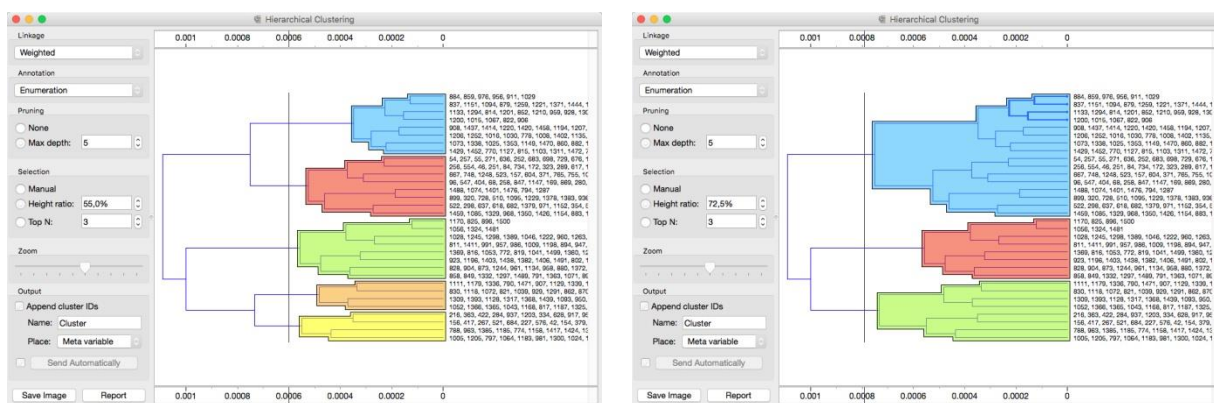
Евклидово расстояние

Манхеттенское расстояние

Рис. 1.10. Иерархическая кластеризация для разных мер расстояния

Также настройки виджета **Hierarchical Clustering** позволяют задать способ выбора уровня кластеризации (блок **Selection** на рис. 1.11).

Результат работы виджета **Hierarchical Clustering** также можно отобразить на точечной диаграмме (**Scatter Plot**). Для этого нужно добавить соответствующий виджет и соединить его с виджетом **Hierarchical Clustering**.



Пять кластеров

Три кластера

Рис. 1.11. Сравнение выбора уровня кластеризации виджета **Hierarchical Clustering**

Порядок выполнения работы

1. Получить варианты заданий.
2. Открыть программу Orange.
3. Загрузить исходные данные с помощью виджета **File**.
4. Осуществить необходимую фильтрацию/очистку данных с помощью виджетов **Select Rows** и **Select Columns** в соответствии с заданием.
5. Осуществить кластеризацию подготовленных данных с помощью алгоритма *k*-средних (*k*-means) с фиксированным и опциональным количеством кластеров.
6. Вывести результаты с помощью точечной диаграммы (виджет **Scatter Plot**) для всех экспериментов в ходе выполнения п. 5.
7. Осуществить кластеризацию подготовленных данных с помощью алгоритма иерархической кластеризации (виджет **Hierarchical Clustering**). Использовать в качестве меры расстояния евклидово, манхеттенское и любое другое расстояние на выбор.
8. Вывести результаты с помощью точечной диаграммы (виджет **Scatter Plot**) для всех экспериментов в ходе выполнения п. 7.
9. Сравнить результаты работы двух алгоритмов (*k*-средних и иерархической кластеризации). Сделать выводы.
10. Отключить фильтрацию данных и повторить эксперимент (только для оптимальных параметров). Сравнить полученные диаграммы с предыдущими. Сделать выводы о влиянии фильтрации (очистки) данных на конечный результат.

11. Подготовить отчёт.

Отчет по работе

1. Титульный лист.
2. Цель работы.
3. Скриншот настроек виджета **Select Rows**.
4. Описание поиска оптимальных параметров алгоритма k-Means.
5. Скриншот основного рабочего листа «Orange» со всеми используемыми виджетами.
6. Скриншот настроек виджета **k-Means**.
7. Описание поиска оптимальных параметров алгоритма иерархической кластеризации.
8. Скриншот настроек виджета **Hierarchical Clustering**.
9. Полученные диаграммы распределения для всех используемых алгоритмов кластеризации и их параметров.
10. Сравнение результатов с включенной и выключенной очисткой данных.
11. Выводы.

Контрольные вопросы и задания

1. Какие типы атрибутов данных вам известны. Приведите примеры.
2. Что такое очистка данных?
3. Как очистка данных может влиять на полученные результаты? Приведите примеры правильной и неправильной очистки/подготовки данных.
4. Что такое кластеризация? Чем кластеризация отличается от классификации?
5. Опишите иерархические алгоритмы кластеризации.
6. Опишите основные алгоритмы квадратичной ошибки.
7. Опишите графовые алгоритмы кластеризации.
8. Классифицируйте алгоритмы кластеризации.
9. Приведите пример применения кластеризации на реальных данных.

2. WEB MINING И TEXT MINING

2.1. Web Mining

Под Web Mining принято понимать использование методов интеллектуального анализа данных, полученных в сети Интернет.

В Web Mining выделяют 3 различных составляющих:

1) *анализ веб-контента* (Web Content Mining) – получение знаний на основе анализа содержимого страниц, документов, записей;

2) *анализ веб-структур* (Web Structure Mining) – позволяет обнаружить ранее неизвестные структуры сети Интернет (взаимосвязь между отдельными страницами и ресурсами, поиск страниц сходной тематики и направленности);

3) *анализ использования веб-ресурсов* (Web Usage Mining) – выявление стереотипов поведения пользователей при работе с сетью Интернет, отдельными страницами и элементами интерфейса.

В Data Mining можно выделить несколько характерных этапов:

1. Этап получения (нахождения) входных данных, пригодных для дальнейшего анализа. Этот этап является отдельной задачей, так как поиск в сети Интернет не является строгой и полностью формализованной задачей. Например, попытка найти с помощью стандартных средств поиска информацию *только по фильму* «Сталинград» неизбежно приведёт к появлению «информационного шума» в виде исторических, военных и других данных.

2. Этап предварительной обработки, на котором полученные сырые данные очищаются, преобразуются к необходимой форме/формату. Например, преобразование всех слов к нижнему регистру, удаление слов-паразитов и т.д.

3. Этап построения модели Web Mining. Оптимальный подбор алгоритмов анализа и их параметров для получения максимально правдоподобного результата. Выработка критериев оценки правдоподобия результатов.

4. Этап анализа. После получения результатов предыдущего этапа необходимо провести их анализ. В ряде случаев приходится вырабатывать решения для

корректировки предыдущих этапов и повторения всего процесса анализа с первого шага.

2.2. Интеллектуальный анализ текста. Тональный анализ

Интеллектуальный анализ текста (text mining) является отдельным направлением анализа информации, целью которого является получение знаний и информации из наборов текстовых документов и больших объёмов текста.

Интеллектуальный анализ текста основывается на применении различных методов машинного обучения, алгоритмах обработки естественного языка, анализе тональности текста и т.д.

Целью анализа тональности (sentiment analysis [5]) является нахождение мнений в тексте, определение их свойств и эмоциональной окраски. Например, данное высказывание: «Вашу новую книгу лучше не читать» носит явно отрицательный характер, в то время как «Эту книгу должен прочитать каждый» имеет явно положительный характер.

Анализ тональности обладает практической ценностью в следующих областях:

- социология – анализ мнения выборки людей на то или иное событие, мнение, решение;
- политология – оценка популярности людей, предложений, партий. Формирование понятной и хорошо принимаемой линии поведения;
- маркетинг – выявление пользовательских предпочтений, анализ отзывов, реакция на выход новых продуктов;
- психология – возможность определения общего психологического климата, выявление патологий и депрессивных состояний отдельных людей.

Ввиду сложности работы с естественным языком анализ тональности является сложной и интересной задачей, которая является темой многих исследований. Далее приводятся только основные (далеко не все) подходы к решению анализа тональности текста.

2.2.1. Анализ на основе правил

Данный вид анализа подразумевает наличие двух компонентов: набора правил оценки тональности и алгоритма проверки текста на соответствие конкретному правилу. Как только мы находим фрагмент текста, соответствующий какому-либо правилу, мы можем присвоить ему некоторое значение тональности. Если текст состоит из набора фрагментов с разной тональной окраской, то общее значение выводится как «сумма» (по некоторому алгоритму) всех фрагментов.

Приведем пример простого правила.

Если есть глагол «люблю» и нет отрицаний, то тональность «положительная».

Это правило очень просто реализовать в любом языке программирования и проверить его на следующих двух тестовых предложениях:

1. Люблю грозу в начале мая, когда весенний первый гром... (Ф. И. Тютчев).
2. Я не люблю, когда мне лезут в душу, тем более, когда в неё плюют.

(В. С. Высоцкий).

Данный метод напрямую зависит от богатства набора правил, т.е. является крайне трудозатратным, так как формирование правил трудно поддается формализации и автоматизации. Также определённую сложность представляет проверка текста на соответствие заданным правилам.

Несмотря на эти проблемы, данный метод анализа широко используется на практике, так как даёт точные результаты. Точность данного подхода повышается с ростом объёма исследуемых массивов текста.

2.2.2. Анализ на основе словаря

По аналогии с анализом на основе правил, анализ на основе словаря включает два основных компонента: словарь тональности (тональный словарь) и алгоритм проверки текста по данному словарю. В общем виде процесс выявления тональной окраски можно представить следующим образом:

1. Проверка всех слов текста по словарю с присвоением значения тональности (если слова нет, мы его не учитываем).

2. Вычисление общей тональности текста (алгоритмы могут быть от самых простых в виде процентиля, среднего арифметического до самых сложных в виде настраиваемых классификаторов на основе нейронных сетей).

Анализ текста по правилам и анализ текста на основе словаря часто используются совместно, что в ряде случаев может дать существенно лучший результат.

2.2.3. *Машинное обучение с учителем*

Классическое решение такого рода задач основывается на наличии обучающей выборки и некоторого машинного классификатора (например, нейронная сеть): сначала необходимо обучить классификатор, а затем оперировать исследуемыми данными.

2.2.4. *Машинное обучение без учителя*

Машинное обучение без учителя представляет собой автоматическое (с минимальным набором известных входных параметров) определение тональности на основе поиска внутренних закономерностей, эвристических правил, различных методов лексического и семантического анализа. Оно является наиболее интересным методом с точки зрения научных исследований [6].

2.3. Проблемы тонального анализа

Все приведенные выше методы тонального анализа не обладают высокой точностью, кроме того, они подразумевают ряд упрощений и пренебрежений. Например, лучше сфокусировать метод анализа на наиболее простых и часто используемых синтаксических и лексических конструкциях («я люблю»; «это хорошо»; «это плохо»; «мне нравится»; «мне не нравится»), а сложными – («советовать или нет: вот в чём вопрос»; «недурно, но безвкусно, хотя полезно») пренебречь, так как для большинства людей и особенно пользователей Интернета скорее характерны более простые конструкции.

Также стоит отдельно остановиться на уровнях тональной классификации. Предельно простой случай: хорошо или плохо (положительно или отрицательно). Несмотря на простоту, этого может быть достаточно для ряда исследовательских

задач. Второй, более интересный и близкий к реальности вариант – это 3 уровня тональной классификации: положительная, нейтральная и отрицательная. Здесь мы можем приписывать сложные для анализа случаи к нейтральной группе и тем самым несколько повысить уровень правдоподобия.

Наиболее сложными способами разделения уровней являются разбиение на более чем 3 уровня классификации (например, сильно положительная, положительная, слабо положительная, нейтральная, слабо отрицательная, отрицательная и сильно отрицательная). Понятно, что для такой тонкой классификации необходимо усложнять применяемые методы кратно увеличению числа уровней.

Заметим, что в современных социальных сетях закрепился ещё один способ выражения эмоций с помощью текста: смайлы и тому подобные значки. Ввиду частого их использования в Интернете и конечности множества этих смайлов сейчас очень популярны сравнительно простые способы тонального анализа на основе смайлов [7–8]. Логично, что упрощение средств выражения эмоций сильно упрощает задачу исследователю.

Лабораторная работа №2. ИНТЕЛЛЕКТУАЛЬНЫЙ АНАЛИЗ ТЕКСТА. WEB MINING. ТОНАЛЬНЫЙ АНАЛИЗ ТЕКСТА В ORANGE

Цель и задача работы

Изучить основные методы Web Mining и Text Mining с использованием приложения «Orange Data Mining».

Используя разные методы текстового анализа, проанализировать реакцию пользователей социальной сети Twitter на различные события (согласно полученному заданию). Осуществить поиск закономерностей в текстовых документах.

Пример построения модели Text Mining в Orange

Для получения доступа к инструментам текстового анализа в **Orange** необходимо подключить соответствующее дополнение **Orange3-Text**. Для этого

откройте окно управления дополнениями (**Options > Add-ons**). Выберите дополнение **Orange3-Text** и нажмите кнопку **OK** (рис. 2.1).

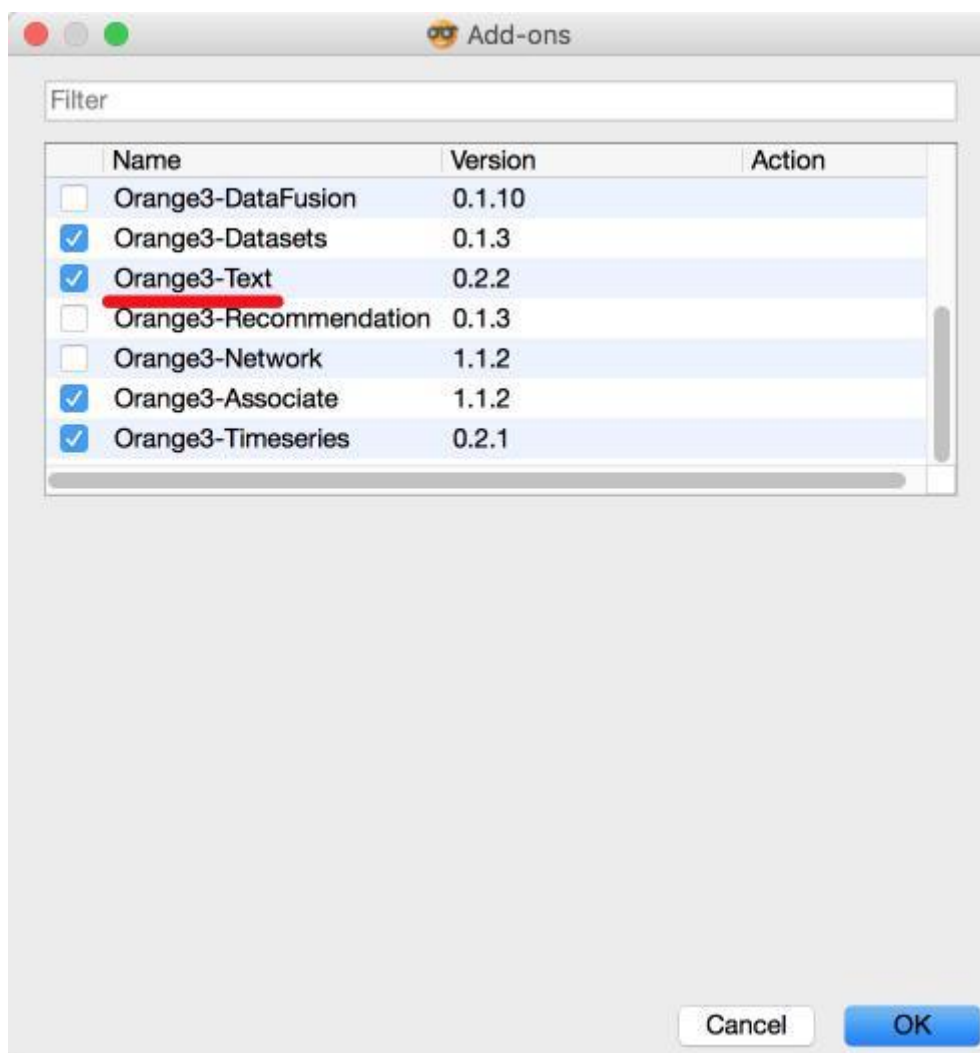


Рис. 2.1. Подключение дополнения текстового анализа Orange3-Text

После этого вам будет предложено перезапустить программу **Orange**. Теперь в списке вкладок будет доступна вкладка **Text Mining**.

Загрузка твитов (виджет Twitter)

В качестве исходных данных для данной лабораторной работы используются сообщения социальной сети Twitter. Для взаимодействия с API Twitter нам понадобится виджет **Twitter** (вкладка **Text Mining**). На рис. 2.2 показано окно настроек данного виджета.

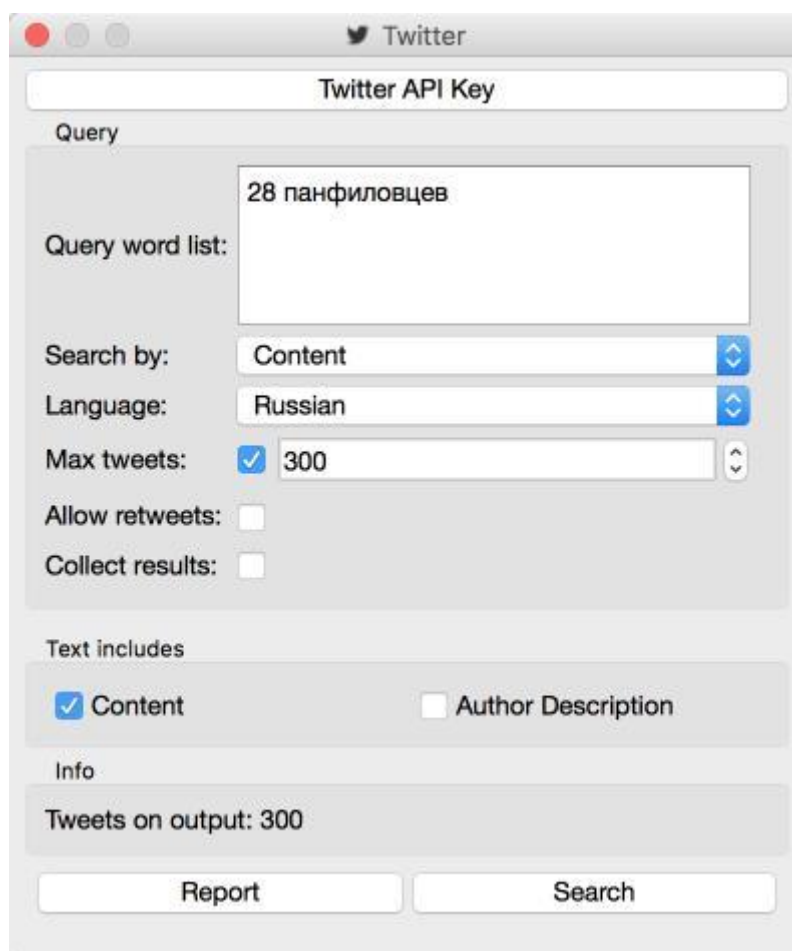


Рис. 2.2. Настройки виджета **Twitter**

Теперь нам необходимо создать приложение Twitter для дальнейшей работы. Для этого нужно зарегистрироваться в социальной сети Twitter и пройти на страницу <https://apps.twitter.com/>. Далее нажать кнопку **Create New App** и ввести короткие сведения о приложении. Пример показан на рис. 2.3. В поле Website можно указать любой адрес, ссылающийся на компьютер пользователя. Например, <http://127.0.0.1:xxxx/>, где xxxx – произвольный номер порта. Значение может быть любым, так как всё равно дальше использоваться не будет.

После этого на вкладке **Keys and Access Tokens** будут доступны данные для доступа к API Twitter. Нам понадобятся API key и API secret. Их необходимо скопировать и ввести в окне настройки виджета **Twitter** (рис. 2.2, кнопка **Twitter API key**).

Далее мы вводим условия поиска сообщений (по одному на каждой строке) в поле **Query word list**. В качестве языка указываем Russian. Максимальное количество сообщений (**Max tweets**) желательно выбрать в диапазоне от 200 до 2000

(малое количество не даст правдоподобных результатов, а очень большое может перегрузить компьютер).

OrangeTextMining

[Details](#)[Settings](#)[Keys and Access Tokens](#)[Permissions](#)

Application Details

Name *

Your application name. This is used to attribute the source of a tweet and in user-facing authorization screens. 32 characters max.

Description *

Your application description, which will be shown in user-facing authorization screens. Between 10 and 200 characters max.

Website *

Your application's publicly accessible home page, where users can go to download, make use of, or find out more information about your app source attribution for tweets created by your application and will be shown in user-facing authorization screens.
(If you don't have a URL yet, just put a placeholder here but remember to change it later.)

Callback URL

Where should we return after successfully authenticating? [OAuth 1.0a](#) applications should explicitly specify their `oauth_callback` URL on the request given here. To restrict your application from using callbacks, leave this field blank.

Рис. 2.3. Создание приложения Twitter

После этого можно нажать кнопку **Search** и посмотреть, сколько сообщений было найдено по предложенному поисковому запросу.

Подготовка сообщений к анализу (виджет Preprocess Text)

Для подготовки данных к дальнейшему анализу необходимо очистить их и привести к нужному виду. Для этого воспользуемся виджетом **Preprocess Text**. На данном этапе мы имеем следующую цепочку (рис. 2.4).

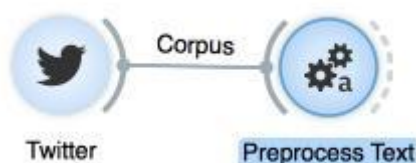


Рис. 2.4. Модель анализа сообщений Twitter

Откроем окно настроек виджета **Preprocess Text** (рис. 2.5). Осуществим преобразование всех слов к нижнему регистру (Lowercase) и удалим из сообщений адреса веб-страниц (Remove urls). В качестве режима лексического анализа (Tokenization) можно выбрать две опции **Regexp** (введенное регулярное выражение будет использовано для выделения токенов) или **Tweet** (предопределённая модель, которая сохранит хэштеги, смайлы и другие специальные символы). Для тонального анализа подходит только режим **Tweet**.

В группе **Filtering** можно задать различные варианты фильтрации и очистки сообщений. Так, можно составить список **Stopwords** (слова, которые будут удалены из сообщений) в виде txt-файла. Можно задать список разрешенных слов (**Lexicon**) также в виде txt-файла. Опция **Regexp** позволяет удалить служебные символы (кавычки, числа, пунктуацию и т.д.). Обязательно используйте регулярные выражения для очистки сообщений.

Поиск топиков (виджеты Bag of Words, Topic Modeling и Word Cloud)

Теперь у нас есть данные, готовые к обработке. Для начала выделим наиболее частые лексические повторения в полученных сообщениях. Для этого нам понадобится добавить три новых виджета:

- bag of Words (позволяет выделить наиболее употребимые, характерные для многих сообщений слова);
- topic Modeling (находит наиболее характерные для всех сообщений наборы слов);
- word Cloud (позволяет отобразить слова в виде облака, где самые характерные слова располагаются максимально близко к центру облака и имеют наибольший размер, а наименее характерные – расположены далеко от центра и имеют минимальный размер).

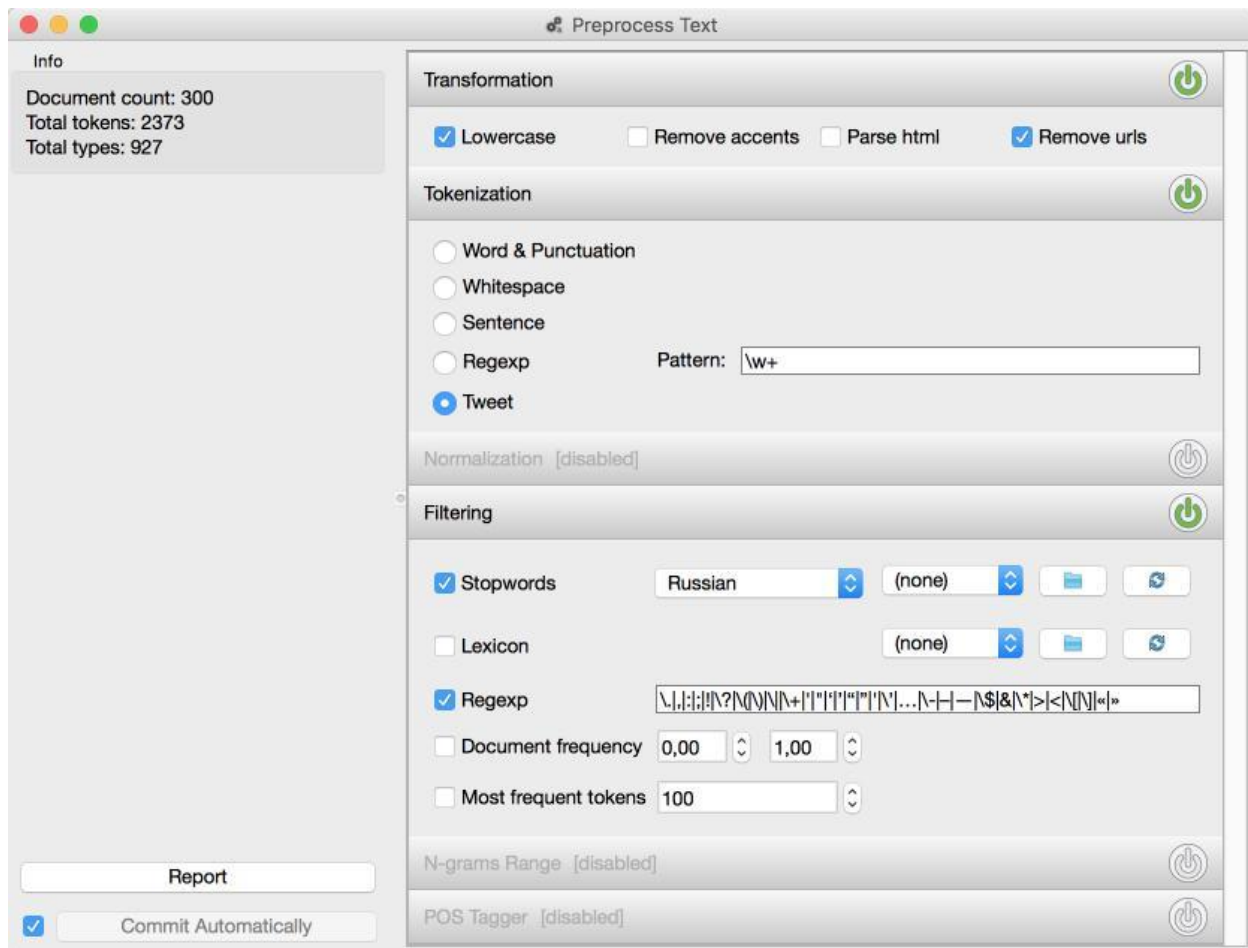


Рис. 2.5. Настройки виджета Preprocess Text

В итоге мы получим следующую модель Text Mining (рис. 2.6).

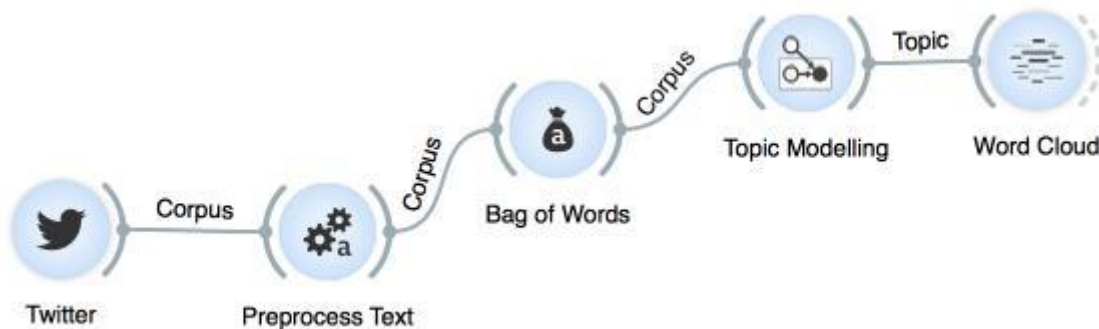


Рис. 2.6. Использование виджетов **Bag of Words**, **Topic Modeling** и **Word Cloud**

Теперь можно открыть окно настроек виджета **Topic Modeling** и посмотреть найденные наборы слов (рис. 2.7). В случае, если в наборах слов встречаются знаки препинания или служебные символы, их необходимо исключить с помощью изменения регулярного выражения для фильтрации виджета **Preprocess Text**, что представлено на рис. 2.5.

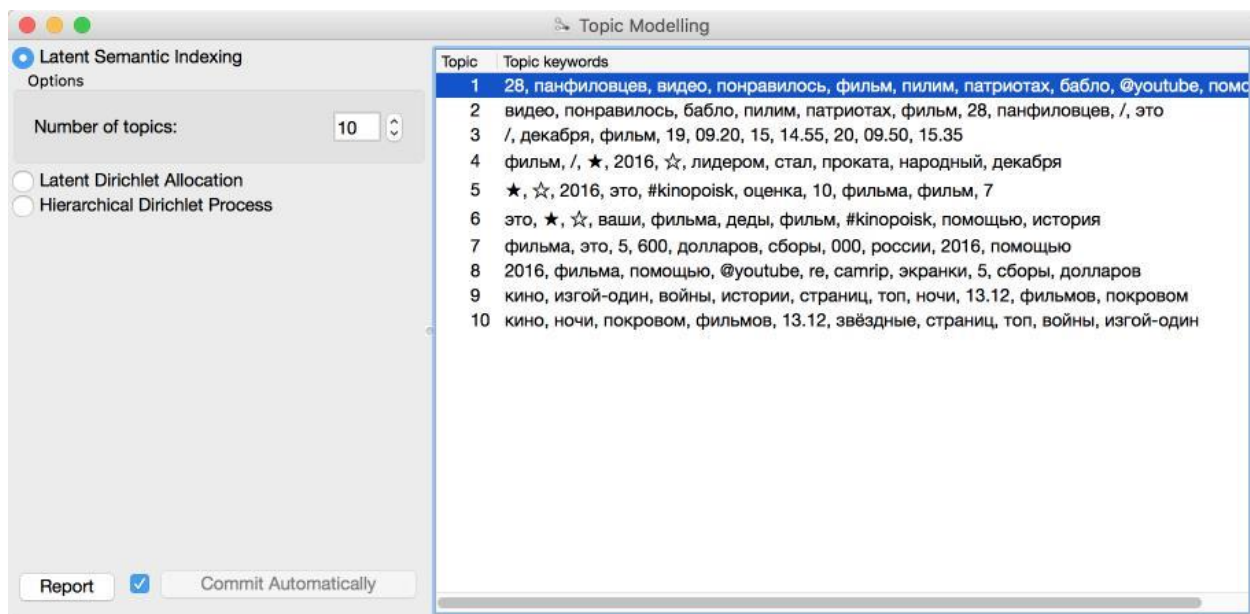


Рис. 2.7. Просмотр найденных наборов слов виджета **Topic Modelling**

Этих данных уже достаточно для некоторого анализа отношения пользователей Twitter к предмету исследования. Для лучшей визуальной картины можно представить полученные результаты в виде облака слов (**Word Cloud**), как показано на рис. 2.8.

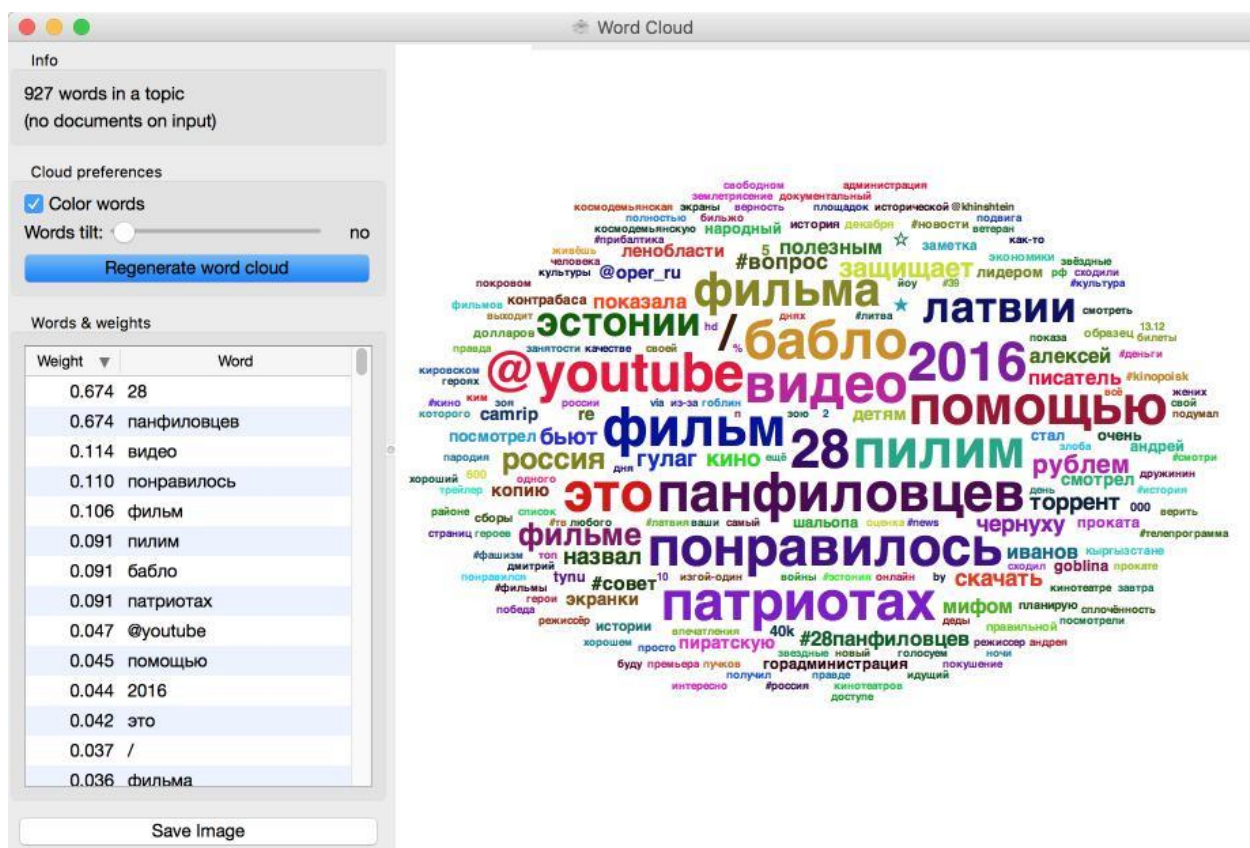


Рис. 2.8. Облако слов

Как видно из полученного облака слов, мы вполне можем улучшить результаты анализа, добавив список **Stopwords**. Например, слова «панфиловцев», «торрент», «28» и некоторые другие не имеют отношения к цели исследования (оценки реакции на фильм), и их можно игнорировать.

Тональный анализ (виджет Tweet Profiler)

Для определения тональной окраски сообщений мы воспользуемся виджетом **Tweet Profiler** (он основан на анализе иконок эмоций или, проще говоря, смайлов). Выходные данные мы представим в виде точечной диаграммы (**Scatter Plot**). Итоговая модель Text Mining выглядит следующим образом (рис. 2.9).

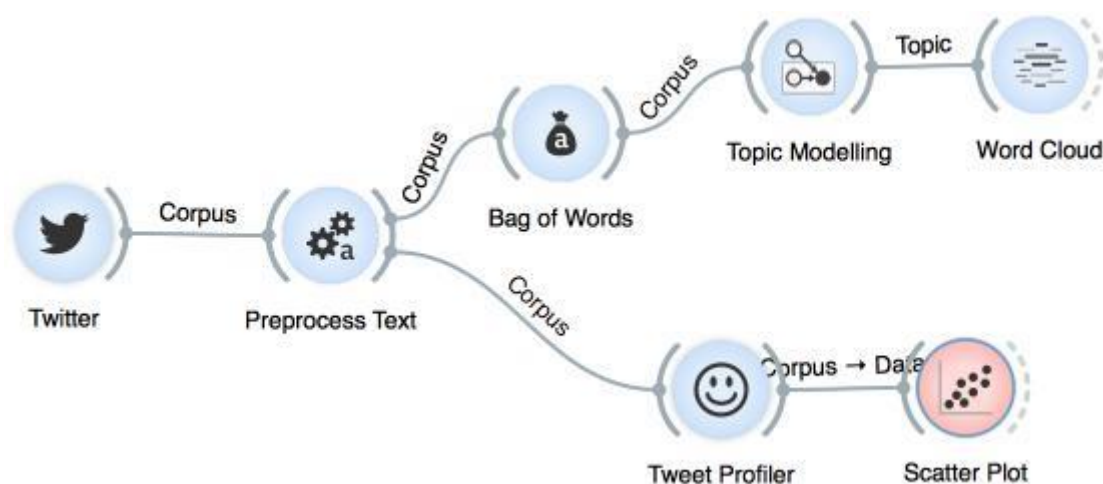


Рис. 2.9. Итоговая модель text mining

Остановимся подробнее на настройках виджета **Tweet Profile** (рис. 2.10). Для нормальной работы данного виджета необходимо получить токен доступа, для этого просто нужно нажать кнопку **Get Token**.

Кроме того, виджет предлагает выбрать шкалу тональности (**Emotions**). Для выбора доступны шкалы Пола Экмана, Роберта Плутчика и современные Профили Настроения (Profile of Mood States или POMS). Попробуйте применить каждую из них и посмотреть на результат.

Для визуализации полученного результата воспользуемся точечной диаграммой. На рис. 2.11 показан пример диаграммы. Для удобства представления по оси x (Axis x) выбираем поле Emotion, а по оси y (Axis y) выбираем поле Date.

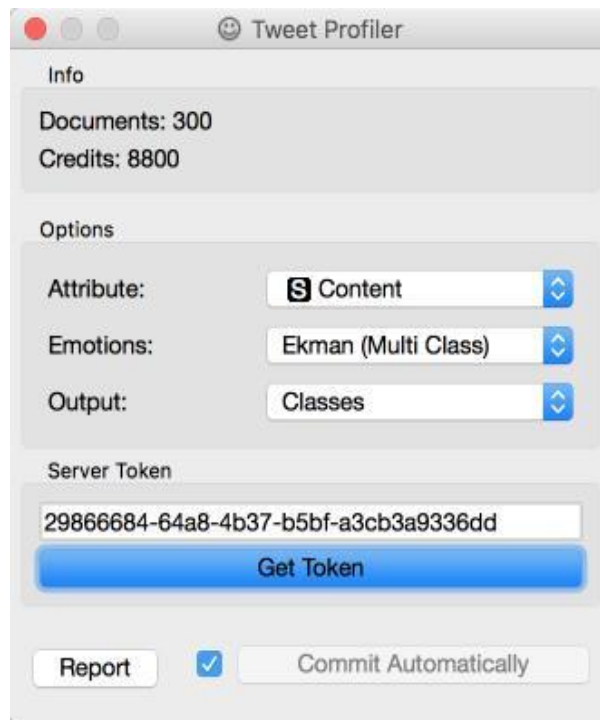


Рис. 2.10. Итоговая модель text mining

Полученная диаграмма является визуальным представлением общей тональной окраски всех полученных сообщений. На основании этой диаграммы можно сделать вывод об общем отношении пользователей к предмету исследования.

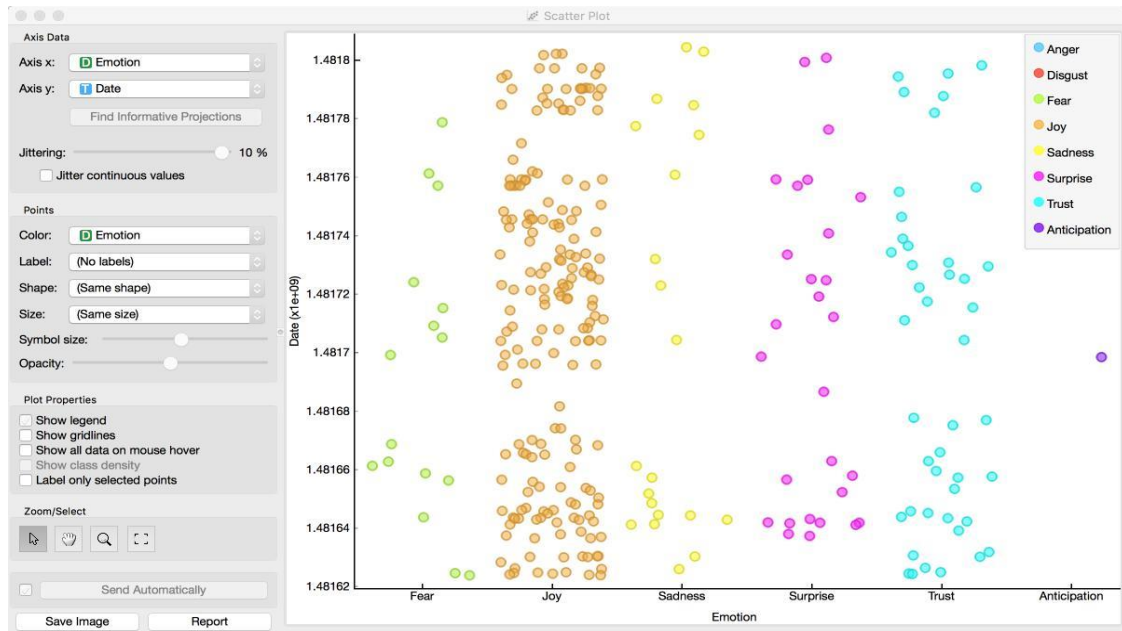


Рис. 2.11. Точечная диаграмма тональности сообщений

Порядок выполнения работы

1. Получить задание.
2. Открыть программу **Orange**.
3. Получить исходные данные с помощью виджета **Twitter**.

4. Осуществить необходимую фильтрацию/очистку данных с помощью виджета **Preprocess Text**.

5. Осуществить поиск топики с помощью виджетов **Bag of Words** и **Topic Modeling**.

6. Вывести результаты с помощью виджета **Word Cloud**.

7. Осуществить тональный анализ данных с помощью виджета **Tweet Profiler**.

8. Вывести результаты с помощью точечной диаграммы (виджет **Scatter Plot**).

9. Сравнить результаты тонального анализа и поиска топики. Сделать выводы.

10. Выбрать фильмы/личности/события с противоположной эмоциональной окраской согласно варианту и осуществить тональный анализ.

11. Найти существующие закономерности в твитах, объяснить их, вывести самостоятельно с помощью цепочки виджетов.

12. Подготовить отчёт.

Отчет по работе

1. Титульный лист.

2. Цель работы.

3. Подробное описание хода выполнения лабораторной работы.

4. Скриншот основного рабочего листа **Orange** со всеми используемыми виджетами.

5. Скриншот настроек виджета **Preprocess Text**.

6. Скриншот настроек виджета **Topic Modeling**.

7. Скриншот настроек виджета **Tweet Profiler**.

8. Полученные диаграммы распределения тональной окраски сообщений.

9. Сравнение результатов с включенной и выключенной очисткой данных.

10. Скриншоты выполненного второго задания (согласно варианту) с противоположными эмоциональными окрасками.

11. Скриншоты и аналитическое объяснение полученных закономерностей в твитах.

12. Выводы.

Контрольные вопросы и задания

1. Что такое Web Mining?
2. Опишите подзадачи Web Mining.
3. Охарактеризуйте этапы Data Mining.
4. Охарактеризуйте интеллектуальный анализ текста.
5. Что такое анализ тональности текста? В чем он заключается? Приведите примеры и опишите его практическое применение.
6. Какие методы анализа тональности текста существуют?
7. Опишите анализ на основе правил, какие компоненты он включает?
8. Назовите виды машинного обучения. В чем их отличия?
9. Назовите проблемы тонального анализа.

3. КЛАССИФИКАЦИЯ

3.1. Понятие классификации

Под задачей классификации понимают необходимость определения, к какому из заранее известных классов (групп) относятся исследуемые объекты [9].

С задачами классификации мы сталкиваемся даже чаще, чем нам может показаться на первый взгляд. Например, мы говорим, что еда вкусная или невкусная, тем самым разделяя её на два класса с противоположными оценками. Всё многообразие вилок, ложек и ножей мы также можем разделить на разные классы (правда, иногда даже искушенному человеку сложно отличить десертную ложку от ложки для мороженого или даже от обычной столовой ложки). Примеры классификации из повседневной жизни можно продолжать бесконечно, однако нас интересует процесс классификации с точки зрения машинного обучения и Data Mining.

Попытки формализации процесса классификации связаны с рядом сложностей. Например, ниже приведены фотографии различных чайников (рис. 3.1).

Как видно, человеку не составит труда классифицировать изображения и понять, что, несмотря на разные размеры, форму, цвет и причудливый вид, это всё равно чайники.

Теперь предположим, что выполнять (простую для человека) задачу классификации предметов на два класса: «чайник» и «не чайник» нужно автоматически. Если задаться вопросом, почему этот конкретный предмет именно чайник, а не что-то другое, мы придём к необходимости описать некоторые общие для всех чайников свойства (параметры). Для этого нам полезно обобщить задачу классификации применительно к машинному обучению и Data Mining.

В Data Mining задачу классификации удобно рассматривать как задачу определения одного из параметров исследуемого объекта на основании анализа значений других параметров. Определяемый параметр (в нашем примере «чайник»/«не чайник») называют *зависимой* переменной, или *целевой* (поэтому в Orange существует тип переменной *target*). Параметры объекта, участвующие в

определении зависимой переменной, называют независимыми переменными (в Orange такие переменные называют *feature* – особенность или характерная черта).



Рис. 3.1. Различные чайники

Как видно, представленный пример имеет очевидную проблему: крайне трудно выделить некоторые общие для всех чайников свойства и в то же время такие, которые будет сравнительно просто формально описать (попробуйте описать такие свойства в качестве упражнения).

3.2. Ирисы Фишера

Проблема выбора хорошего (наглядного, показательного и сравнительно простого) примера классификации не нова. Например, ещё в 1936 г. английский учёный Рональд Фишер для описания разработанного им метода дискриминантного анализа использовал в качестве исследуемого набора данных классификацию ирисов [10]. С тех пор этот набор данных является классическим и очень часто

используется для демонстраций и примеров. Orange также не стал исключением, и вы можете найти файл *iris.tab* в примерах данных, поставляемых вместе с данной программой.

Рассмотрим данный набор данных подробнее применительно к машинному обучению. Итак, Фишер предлагал использовать в качестве независимых переменных следующие свойства ирисов:

- sepal length – длина чашелистника;
- sepal width – ширина чашелистника;
- petal length – длина лепестка;
- petal width – ширина лепестка.

Для человека, далёкого от ботаники, эти понятия мало информативны, однако их особенность заключается в том, что каждый из параметров является числом (длиной). Фишер использовал ирисы трёх видов – ирис щетинистый (*Iris setosa*), ирис виргинский (*Iris virginica*) и ирис разноцветный (*Iris versicolor*) [10]. На рис. 3.2 приведены фотографии данных ирисов.



а



б



в

Рис. 3.2. Ирисы Фишера: а – ирис щетинистый;
б – ирис виргинский; в – ирис разноцветный

Ниже приведён пример этого тестового набора для каждого вида ирисов (табл. 1).

Теперь задача о классификации ирисов может быть достаточно легко формализована: нам нужно найти такую функцию от четырёх независимых переменных, чтобы она предсказывала вид ириса с минимальной ошибкой. Под ошибкой понимается неверное определение вида ириса.

$$f(\text{sepalLength}, \text{sepalWidth}, \text{petalLength}, \text{petalWidth}) = [\text{setosa}|\text{versicolor}|\text{virginica}]$$

Пример данных ирисов Фишера

Независимые переменные (<i>features</i>)				Зависимая переменная (<i>target</i>)
Sepal length	Sepal width	Petal length	Petal width	Вид ириса
5.1	3.5	1.4	0.2	Setosa
5.0	3.4	1.5	0.2	Setosa
5.5	2.3	4.0	1.3	Versicolor
6.5	2.8	4.6	1.5	Versicolor
6.3	2.7	4.9	1.8	Virginica
6.7	3.3	5.7	2.1	Virginica

Функцию, которая по набору независимых переменных определяет значение зависимой переменной, называют *классификатором*. Сразу скажем, что здесь речь идёт не обязательно о какой-то математической функции, а скорее о функции в смысле программирования (когда на вход мы подаём некоторые значения, а на выходе получаем результат).

3.3. Процесс классификации

Теперь мы можем описать процесс решения задачи классификации по пунктам.

1. Сформировать обучающий набор данных (training set).
2. Подготовить формальное описание объекта (определить все независимые переменные).
3. Определить возможные значения зависимой переменной (сформировать множество возможных классов).
4. Найти оптимальный классификатор для подготовленного обучающего набора (как правило, приходится сравнивать качество работы нескольких алгоритмов классификации, чтобы выбрать наилучший).

5. В случае достижения заданных параметров точности классификации обученный классификатор можно использовать для классификации объектов неизвестных классов.

Если заданная точность не была достигнута, то необходимо пересмотреть пп. 1–4. Часто можно изменить модель объекта исследования. Так, усовершенствовать набор независимых переменных можно путём выявления и включения новых (существенных) свойств или, наоборот, исключением шумовых свойств, которые в действительности не влияют на определение класса.

Приведём пример. Допустим, решается задача определения группы риска возникновения сердечно-сосудистых заболеваний у пациентов. В этом случае имеются некоторые медицинские показатели с рядом числовых и списочных (категорийных) значений (например, кардиограмма, анализ крови и т.д.), и нужно составить первоначальную модель исследуемого пациента только на основе этих данных. Понятно, что результаты классификации в данном случае могут зависеть и от некоторых других свойств пациента, таких как возраст, пол, наследственность, систематические занятия спортом и т.д. В то же время такие показатели, как цвет глаз, вряд ли будут полезными. Данный пример показывает, что классификации являются сложным процессом, требующим комплексного подхода.

Другой частой проблемой при работе с реальными данными является низкое качество исходных данных, в которых могут встречаться ошибочные или пропущенные значения. Для минимизации влияния этих факторов необходимо проводить очистку данных (например, исключение заведомо ошибочных и противоречивых значений или полное исключение записи в случае неопределённости ключевых свойств).

Также отдельно выделяют проблемы построения классификаторов, самыми известными из которых являются *overfitting* (переобучение) и *underfitting* (недостаточное обучение) [9]. Под переобучением понимают такую ситуацию, когда полученный классификатор отлично работает на обучающей выборке, но крайне плохо работает с неизвестными ему данными (это часто случается при попытке классификатора привести аномальные или ошибочные значения к общим

закономерностям). Ситуация недостаточного обучения, напротив, показывает неспособность классификатора решить поставленную задачу (т.е. количество ошибочных ответов сильно превышает допустимые значения). Это возможно, когда подготовленные данные не содержат внутренних закономерностей или для их выявления нужно менять метод обнаружения и модель.

3.4. Алгоритмы классификации

3.4.1. *Дерево решений*

Дерево решений – это наглядный способ представления правил классификации в виде строгой иерархической, последовательной структуры, визуально похожей на дерево [9].

«Листьями» дерева решений являются значения зависимой переменной (классы). «Ветвями» (рёбрами) являются переходы между узлами и/или листьями в зависимости от результата проверки в узле. Узел, в свою очередь, – это проверка какой-либо независимой переменной или нескольких переменных на удовлетворение условию (это может быть простое сравнение или сложная математическая функция). На рис. 3.3 приведён простой пример дерева решений для классификации оценки студента за одну лабораторную работу.

Овалом обозначен узел, линией – ребро, прямоугольником – лист.

Объект принадлежит конкретному классу (листу), если его независимые свойства удовлетворяют условиям, записанным в узлах дерева на пути от корневого узла к листу, соответствующему этому классу.



Рис. 3.3. Дерево решений для оценки по лабораторной работе

У дерева решений существуют некоторые проблемы, например, если какая-либо независимая переменная не имеет значения, то определить путь становится невозможным. Представим, что в приведенном выше примере одна из лабораторных работ не содержит требований к отчёту, тогда получается, что мы попадаем в ситуацию неопределённости. Понятно, что выход из таких ситуаций может быть найден, например, заменой пропущенного значения на значение по умолчанию или выбором той ветви, которая имеет большее количество листьев. В примере с отсутствием требований к отчёту легко понять, что нужно сразу переходить к контрольным вопросам (или другими словами, считать работу соответствующей требованиям по умолчанию). Также можно обойти такую тупиковую ситуацию добавлением специальной ветви для пропущенного значения.

Деревья решений хорошо подходят для объектов с большим количеством категориальных свойств или с числовыми параметрами, значения которых легко группируются или представляют небольшое множество (например, 0 и 1 или шкалу от 0 до 5 как в нашем примере).

3.4.2. Наивный байесовский классификатор

В основе наивного классификатора Байеса лежит расчёт вероятности принадлежности объекта к классу и предположение, что все независимые переменные действительно никак не зависят друг от друга (что и является весьма «наивным» предположением, отсюда и получилось название данного алгоритма). Хотя в реальной жизни объекты с действительно независимыми свойствами сложно найти, наивный классификатор Байеса часто даёт удовлетворительные результаты, особенно для небольших наборов данных.

Обозначим вероятность того, что некоторый объект i_j принадлежит классу c_r как $P(y = c_r)$. Смысл алгоритма в том, чтобы рассчитать условную вероятность вхождения объекта в класс c_r при равенстве независимых переменных определённым значениям. Данную вероятность можно вычислить с помощью аппарата теории вероятности по формуле

$$P(y = c_r | E) = \frac{P(E | y = c_r) \times P(y = c_r)}{P(E)}. \quad (3.1)$$

В (3.1) событие равенства независимых переменных определённым значениям обозначим как E , а вероятность его наступления – $P(E)$ [9]. В результате мы получаем правила, условиями которых являются сравнения всех независимых переменных с возможными значениями, а заключениями – все возможные значения зависимой переменной.

Если $x_1 = c_h^1$ и $x_2 = c_h^2$ и ... $x_m = c_h^m$, тогда $y = c_r$.

Предполагая, что все переменные не зависят друг от друга, мы можем выразить вероятность $P(E | y = c_r)$ через произведение вероятностей для каждой независимой переменной:

$$P(E | y = c_r) = P(x_1 = c_p^1 | y = c_r) \times P(x_2 = c_d^2 | y = c_r) \times \dots \times P(x_m = c_b^m | y = c_r).$$

Тогда вероятность для всего правила можно вычислить по формуле

$$P(y = c_r | E) = P(x_1 = c_p^1 | y = c_r) \times P(x_2 = c_d^2 | y = c_r) \times \dots \times \\ \times P(x_m = c_b^m | y = c_r) \times P(y = c_r) / P(E).$$

Теперь вероятность вхождения объекта в класс c_r при условии равенства его независимой переменной x_h значению c_p^1 можно определить по формуле

$$P(x_h = c_d^h | y = c_r) = P(x_h = c_d^h \text{ и } y = c_r) / P(y = c_r).$$

Другими словами, это отношение количества объектов класса c_r , для которых справедливо равенство $x_h = c_d^h$ к общему количеству объектов класса c_r .

Получив в результате вероятности вхождения исследуемого объекта в разные классы, мы выбираем класс с наибольшей вероятностью в качестве ответа.

У данного алгоритма существует проблема: что делать, если в обучающем наборе данных нет ни одного объекта, имеющего значение c_d^h переменной x_h , но относящегося при этом к классу c_r ? Получается, что вероятность для данной переменной будет равна нулю, следовательно, и полная вероятность соответствующего правила также будет равна нулю. Для решения этой проблемы используют оценочную функцию Лапласа [9]. Суть решения в том, чтобы к каждой вероятности (в том числе и нулевой) прибавить некоторое ненулевое значение.

Одним из примеров использования данного классификатора можно считать отнесение электронных писем к спаму на основе частоты встречающихся слов, характерных для спама: купи, скидка, специальное предложение, подарок и т.д.

3.4.3. Метод опорных векторов (Support Vector Machines)

Данный алгоритм был предложен Вапником и Червоненкисом [9] в 1974 г. Алгоритм опорных векторов считается эффективным и находит широкое применение в задачах классификации.

Идея метода основана на предположении, что лучшим способом разделения точек в m -мерном пространстве является $m-1$ плоскость, заданная функцией $f(x)$. Эта плоскость должна быть равноудалена от ближайших соседних точек разных классов. На рис. 3.4 представлена графическая интерпретация данного метода для двумерного пространства.

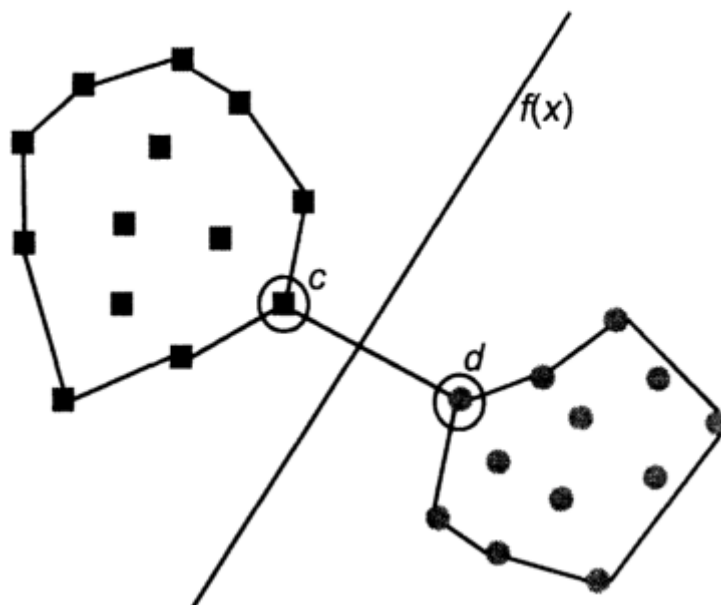


Рис. 3.4. Метод опорных векторов для двумерного пространства

Для данного примера решением будет являться плоскость, равноудалённая от соседних точек разных классов c и d . Данный метод интерпретирует объекты и их свойства как векторы размера m . Независимые переменные (свойства объекта) в этом случае выступают как координаты векторов. Ближайшие векторы разных классов называют векторами поддержки (support vectors) – отсюда и происходит название метода.

Алгоритм опорных векторов исходит из того соображения, что ошибка классификатора тем меньше, чем больше мера расстояния между разделяющей плоскостью и ближайшими векторами соседних классов.

Одним из важных свойств метода опорных векторов является непрерывное уменьшение ошибки классификации вследствие постоянного уточнения формы разделяющей плоскости.

3.4.4. Метрические классификаторы

Если для исследуемых данных можно ввести понятие *расстояния* между разными объектами, то для классификации применимы так называемые метрические классификаторы.

Обозначим расстояние от исследуемого объекта u до объекта x_i как $d(u, x_i)$. Для расстояния d определим весовую функцию $w(d(u, x_i))$, которая показывает значимость данного расстояния, например, в самом простом виде: чем ближе

объекты, тем больше значение весовой функции, и наоборот, чем дальше объекты друг от друга, тем меньше.

Далее для каждого известного класса вычисляется суммарный вес расстояний до всех соседей; соседи – это известные объекты данного класса. Объект u будет отнесён к тому классу, для которого суммарный вес будет максимальным.

Понятно, что задать меру расстояния d и весовую функцию w можно разными способами, поэтому метрические классификаторы часто различают как раз по способу определения веса и расстояния. Рассмотрим далее только метод ближайшего соседа и метод k ближайших соседей для определения весов.

В программе **Orange** можно найти виджет **Nearest Neighbors**, в котором в качестве опции можно выбрать разные меры расстояний и весовые функции. Так, доступны евклидово, манхэттенское расстояния и расстояние Чебышева.

Метрические классификаторы относят к методам рассуждения по прецедентам (case-based reasoning). Действительно, всегда можно сказать, что данный объект входит в некоторый класс, потому что есть похожие на него (соседние) объекты этого класса.

3.4.4.1. Метод ближайшего соседа (nearest neighbor)

Самый простой способ задать весовую функцию – это найти самый близкий (похожий) объект. К какому классу относится этот объект, к такому и отнесём объект исследуемый.

Реализовать данный алгоритм очень просто: достаточно перебрать все известные объекты и для каждого посчитать расстояние до исследуемого объекта; класс объекта с минимальным расстоянием и будет искомым.

Очевидно, что данный подход хорош только простотой реализации и приемлемо работает для хорошо разделимых объектов. Если, например, в обучающей выборке встречаются перекрытия или аномалии, то правильная классификация будет затруднительна. Поскольку весовая функция, по существу, отсутствует (вернее сказать, полностью зависит от выбранной функции расстояния), то мы также не можем влиять на качество классификации путём подбора её параметров.

3.4.4.2. Метод k ближайших соседей (k nearest neighbors)

Данный метод берёт в расчёт не один ближайший объект, а некоторую совокупность. Мы будем относить исследуемый объект к тому классу, представителей которого окажется больше среди k ближайших соседей.

Понятно, что если $k = 1$, то данный метод вырождается в предыдущий (один ближайший сосед) и будет иметь указанные выше недостатки. Также если k принять равным длине всей выборки, то мы получим слишком устойчивый классификатор.

Тогда разумно задаться вопросом: как выбрать оптимальное значение k ? Для этого существует специальный критерий контроля с исключением объектов по очереди (он присутствует в **Orange** под названием «leave one out» в виджете **Test & Score**). Суть состоит в том, чтобы найти такое количество k соседей, которые правильно классифицируют исключённый объект.

Возможна и чуть более сложная вариация данного алгоритма, в которой оценивается среднее расстояние до k ближайших соседей (а не их количество).

Также в данный алгоритм можно внести некоторую нелинейную весовую функцию, чтобы исключить вероятность возникновения равенства количества ближайших соседей из нескольких классов.

3.4.5. Случайный лес

Данный алгоритм был сформулирован Лео Брейманом в 2001 г. [11]. Метод основан на построении некоторой совокупности (ансамбля) деревьев решений. Каждое дерево строится не по полной выборке объектов, а по некоторой её части, причём всегда учитывается разное (случайное) количество независимых переменных.

Классификация осуществляется с помощью голосования классификаторов, определяемых отдельными деревьями. Известно, что точность (вероятность корректной классификации) ансамблей классификаторов существенно зависит от разнообразия (diversity) классификаторов, составляющих ансамбль или, другими словами, от того, насколько коррелированы их решения. Точнее, чем более разнообразны классификаторы ансамбля (корреляция их решений стремится к нулю), тем выше вероятность правильной классификации [12].

В случайных лесах решения составляющих их деревьев слабо коррелированы вследствие двойной «инъекции случайности» в алгоритм построения случайного леса – на стадии усечения выборки и на стадии случайного отбора независимых переменных.

Данный метод очень популярен вследствие того, что он обладает рядом достоинств:

- метод гарантирует защиту от переобучения даже в случае, когда количество признаков значительно превышает количество наблюдений. Это свойство выделяет данный алгоритм среди других методов классификации;
- для построения случайного леса по обучающей выборке требуется задание всего двух параметров, которые требуют минимальной настройки: это количество случайно выбранных объектов и количество случайно выбранных независимых переменных;
- метод Out-Of-Bag (OOB), предложенный Брейманом [11], обеспечивает получение естественной оценки вероятности ошибочной классификации случайных лесов на основе наблюдений, не входящих в обучающие выборки, используемые для построения деревьев (эти наблюдения называются OOB выборками);
- случайные леса могут использоваться не только для задач классификации, но и для задач выявления наиболее информативных признаков, кластеризации, выделения аномальных наблюдений и определения прототипов классов;
- обучающая выборка для построения случайного леса может содержать признаки, измеренные в разных шкалах: числовой и категориальной, что недопустимо для многих других классификаторов;
- случайный лес легко масштабируется, что является неоспоримым преимуществом при обработке больших объёмов данных.

3.5. Точность классификации: оценка уровня ошибок

Выше были описаны некоторые алгоритмы классификации и теперь поставим задачу оценить качество их работы. Для оценки качества нам нужно решить две подзадачи.

1. Определить численные показатели качества классификации.

2. Научиться вычислять эти показатели, используя имеющиеся наборы данных.

Численные показатели будут подробно рассмотрены далее. Сейчас остановимся на второй подзадаче.

Стоит отметить, что все алгоритмы классификации требуют двух наборов данных: обучающего и тестового. Очевидно также, что в идеале чем больше вариаций таких наборов данных мы будем иметь, тем точнее мы сможем определить качество работы того или иного алгоритма. Проблема состоит в том, что подготавливать такие наборы не всегда удобно. Как правило, приходится работать с одним набором данных, который мы собрали самостоятельно или получили каким-либо образом. Теперь нам нужно из одного набора сделать как минимум два: обучающий и тестовый. Очевидно, нам просто нужно разделить имеющийся набор на две части, вопрос лишь в том, как это сделать.

Существует три основных подхода решения данной проблемы.

1. Разбить все записи на n групп (folds) и по очереди считать одну из групп тестовой, а остальные $(n-1)$ группы будут обучающей выборкой. Таким образом, мы проверим алгоритм n раз, и все объекты исходного набора побывают как в обучающих, так и в тестовых наборах.

2. Аналогично первому, только $n = N$, где N – количество объектов в выборке. Этот подход хорош для небольших наборов данных и крайне медленно работает для больших наборов, так как в реальных задачах N может измеряться десятками миллионов.

3. Установить процентное соотношение между обучающей и тестовой выборками и случайным образом выбирать указанные доли из исходного набора указанное число раз. Например, мы решили, что 20 % объектов будет тестовым набором, а остальные 80 % – обучающим. Поскольку мы выбираем объекты случайным образом, мы можем повторять это разбиение сколь угодно много раз.

Имея обучающую и тестовую выборки, мы можем провести так называемую кросс-проверку (cross-validation). Эта процедура состоит в том, что точность

классификации тестового множества сравнивается с точностью классификации обучающего множества. Если эти результаты примерно равны, то проверка считается пройденной.

3.6. Численная оценка качества алгоритма

3.6.1. Точность (Accuracy)

Самым простым численным показателем качества классификации может быть отношение правильно классифицированных объектов к общему их количеству. Такую характеристику называют *точностью* (ассигасу), которая задается формулой

$$Accuracy = \frac{P}{N}, \quad (3.2)$$

где P – количество документов, по которым классификатор принял правильное решение, а N – размер выборки в (3.2). Понятно, что идеальный классификатор имеет точность, равную единице.

Эта метрика имеет один существенный недостаток: она не учитывает распределение классов в обучающей выборке. То есть если для каких-то классов в выборке существенно больше данных, то для них классификатор будет работать отлично, а для классов с меньшей долей информации – хуже.

Например, точность классификатора равна 0,9 (что очень хорошо), но в рамках какого-то одного класса точность может быть менее 0,3.

Конечно, можно искусственно сбалансировать обучающую выборку, выровняв соотношения классов (например, для четырёх классов в обучающей выборке должно быть примерно по 25 % объектов каждого класса). Для проведения такой операции нужно определить класс с наименьшим числом представителей и взять столько же представителей других классов. Минусом такого подхода является вмешательство в исходный набор данных, так как информация о доле объектов каждого класса сильно исказится. Поэтому точность хоть и используется для оценки качества, но должна быть либо подтверждена другими характеристиками, либо оценена адекватно структуре обучающих данных.

3.6.2. Точность и полнота

Точность (precision) и полнота (recall) являются ключевыми показателями качества классификации (и не только классификации). Чаще всего они используются сами по себе, но иногда включаются в качестве основы для производных метрик, таких как F -мера или R -Precision.

Точностью в пределах класса называют отношение правильно классифицированных объектов к общему числу объектов, которые классификатор отнёс к этому классу. Она определяется по формуле

$$Precision = \frac{TP}{TP + FP}, \quad (3.3)$$

где T_P – истинно положительные решения; F_P – ложноположительные решения в (3.3).

Полнота определяется как доля найденных классификатором объектов некоторого класса относительно всех объектов данного класса в тестовом наборе по формуле

$$Recall = \frac{TP}{TP + FN}, \quad (3.4)$$

где F_N – ложноотрицательные решения в (3.4).

Приведём пример. Пусть классификатор должен определить, есть ли на изображении автомобиль или нет. У нас есть 100 изображений, и на 50 из них действительно изображён автомобиль. На выходе классификатора мы получили следующие результаты: 40 изображений были отнесены к классу «автомобиль», причём правильными из них было 30. Итого мы имеем:

$T_P = 30$ – истинно положительных решений;

$F_N = 20$ – ложноотрицательных решений (в действительности 50 объектов класса, но правильно отнесено только 30, значит $50 - 30 = 20$);

$F_P = 10$ – ложноположительных решений (40 было отнесено к классу, но только 30 верно, значит $40 - 30 = 10$).

Для класса «автомобиль» мы можем вычислить точность как: $30 / (30 + 10) = 0,75$, и полноту как $30 / (30 + 20) = 0,6$.

3.6.3. Confusion Matrix

Для небольшого количества классов существует наглядный способ оценки качества классификации – матрица неточностей (confusion matrix). Конечно, ограничение в виде небольшого количества классов носит формальный характер, но пользователю вряд ли будет удобно работать с матрицей размерности более 100 элементов.

Матрицей неточностей является матрица размера $N \times N$, где N – это число классов. За строками матрицы можно закрепить истинные значения исходной выборки (заведомо правильная классификация), а за столбцами – результат работы исследуемого классификатора (это также не является ограничением, столбцы и строки можно поменять местами, суть от этого не изменится). Если классификатор отнёс объект к классу m , а его истинный класс – n , то мы увеличиваем счётчик на пересечении столбца n и строки m . Таким образом, легко понять, что если m и n совпадают, то мы попадаем на диагональный элемент, в этом и есть основной смысл матрицы неточностей: идеальный классификатор имеет ненулевую диагональ и остальные элементы, равные нулю.

На рис. 3.5 приведён пример матрицы неточности для шести классов. По столбцам и строкам указаны названия классов (в нашем случае, 1.0, 2.0 и т.д.).

		Predicted						
		1.0	2.0	3.0	5.0	6.0	7.0	Σ
Actual	1.0	49	16	5	0	0	0	70
	2.0	11	58	4	1	1	1	76
	3.0	6	7	4	0	0	0	17
	5.0	0	1	0	11	0	1	13
	6.0	0	2	0	0	5	2	9
	7.0	1	3	0	1	0	24	29
Σ		67	87	13	13	6	28	214

Рис. 3.5. Матрица неточностей для 6 классов

Видно, что классификатор хорошо справился со своей задачей, однако у него возникли проблемы с первыми тремя классами (число неверно классифицированных объектов для них наибольшее).

Имея данную матрицу можно рассчитать точность и полноту для всех классов. Точность будет равна отношению диагонального элемента матрицы к сумме элементов всего столбца класса. Полнота – отношение диагонального элемента матрицы к сумме элементов всей строки класса.

Обобщённую точность и полноту классификатора можно рассчитать как среднее арифметическое этих значений для всех классов.

3.6.4. *F-мера*

Очевидно, что чем ближе точность и полнота к единице, тем лучше выбран и настроен классификатор. Для реальных данных и классификаторов часто нельзя одновременно добиться и высокой точности, и полноты. Для этого ввели производную характеристику, которую называли *F-мера*.

F-мера – это гармоническое среднее между точностью и полнотой [13], представленное формулой

$$F = 2 \times \frac{PRECISION \times RECALL}{PRECISION + RECALL}. \quad (3.5)$$

Среднее гармоническое обладает важным свойством – оно стремится к нулю, если хотя бы один из аргументов также стремится к нулю. Именно поэтому оно является более предпочтительным по сравнению со средним арифметическим (если алгоритм будет относить все объекты к положительному классу, то он будет иметь $recall = 1$ и $precision \ll 1$, а их среднее арифметическое будет больше $1/2$, что недопустимо).

Указанную формулу расчёта (3.5) *F-меры* можно модифицировать, чтобы точность и полнота имели разный вклад. Таким образом, можно рассчитывать *F-меру* с приоритетом точности или с приоритетом полноты.

Формулу расчёта *F-меры* с использованием весового коэффициента можно записать в виде

$$F = (\beta^2 + 1) \times \frac{PRECISION \times RECALL}{\beta^2 PRECISION + RECALL}, \quad (3.6)$$

где β в (3.6) принимает значения в диапазоне $0 < \beta < 1$, если вы хотите отдать приоритет точности, а при $\beta > 1$ приоритет отдается полноте. При $\beta = 1$ формула сводится к предыдущей, и мы получаем сбалансированную F -меру (также ее называют $F1$). Если $\beta^2 = 0,25$, F -мера получится с приоритетом точности, если $\beta^2 = 2$, то получим F -меру с приоритетом полноты.

F -мера является хорошей интегральной характеристикой качества классификации, так как связывает точность и полноту.

3.6.5. ROC-кривая

Прежде чем вводить понятие ROC-кривой (Receiver Operating Characteristic curve), необходимо разобраться с матрицей неточности для бинарного классификатора.

Допустим, у нас есть бинарный классификатор, т.е. допускающий только два решения: истина (1) и ложь (0). Рассмотрим, какие варианты решений мы можем получить на выходе данного классификатора.

1. Классификатор правильно предсказал, что объект относится к классу 0. Назовём такой вариант исхода истинно отрицательным (True-Negative или сокращённо TN). Например, детектор спама не классифицировал письмо от вашего друга как спам.

2. Классификатор неверно предсказал, что объект относится к классу 0, хотя в действительности он относится к классу 1. Назовём такой вариант исхода ложно-отрицательным (False-Negative или сокращённо FN). Например, детектор спама пропустил сообщение, которое является спамом.

3. Классификатор неверно предсказал, что объект относится к классу 1, хотя в действительности относится к классу 0. Назовём такой вариант исхода ложно-положительным (False-Positive или сокращённо FP). Например, детектор спама классифицировал письмо от вашего друга как спам.

4. Классификатор верно предсказал, что объект относится к классу 1. Назовём такой вариант исхода истинно положительным (True-Positive или сокращённо TP). Например, детектор спама правильно обнаружил спам-сообщение.

Обработав все входные данные, мы точно можем сказать (подсчитать) количество каждого варианта исхода. Пример приведён в табл. 2. Как видно, это очень похоже на матрицу неточностей (confusion matrix).

Таблица 2

Матрица неточностей бинарного классификатора

		Предсказанные классы	
		Класс 1	Класс 0
Истинные классы	Класс 1	10 TP	2 FN
	Класс 0	3 FP	35 TN

В этом примере среди пятидесяти объектов входной выборки 45 объектов классифицированы верно, а пять объектов – ошибочно.

Мы можем вычислить две характеристики классификатора на основе приведённой матрицы неточностей (позже мы сведём их в одну).

1. Доля истинно положительных решений (True-Positive Rate или сокращённо TPR). Определим её как $TP / (TP + FN)$. Эта величина показывает отношение правильно классифицированных объектов, относящихся к классу 1, к общему числу объектов этого класса. Понятно, что чем выше TPR, тем меньше неверно предсказанных объектов класса 1.

2. Доля ложноположительных решений (False-Positive Rate или сокращённо FPR). Определим её как $FP / (FP + TN)$. Эта величина показывает отношение неверно классифицированных объектов, отнесённых к классу 1, к общему числу объектов класса 0. Понятно, что чем выше FPR, тем больше объектов класса 0 мы неверно предсказали.

Чтобы объединить две эти характеристики в одну, мы сначала вычисляем TPR и FPR для логистической регрессии с разными значениями порога от 0 до 1 (например, 0,0; 0,1; 0,2 ... 1,0), затем отмечаем полученные значения на графике, причём по оси x будут значения FPR, а по оси y – TPR.

Полученная кривая называется ROC-кривой, а основной характеристикой качества классификации будет выступать площадь под этой кривой (Area Under the ROC Curve). ROC-кривая для нашего примера выглядит следующим образом (рис. 3.6).

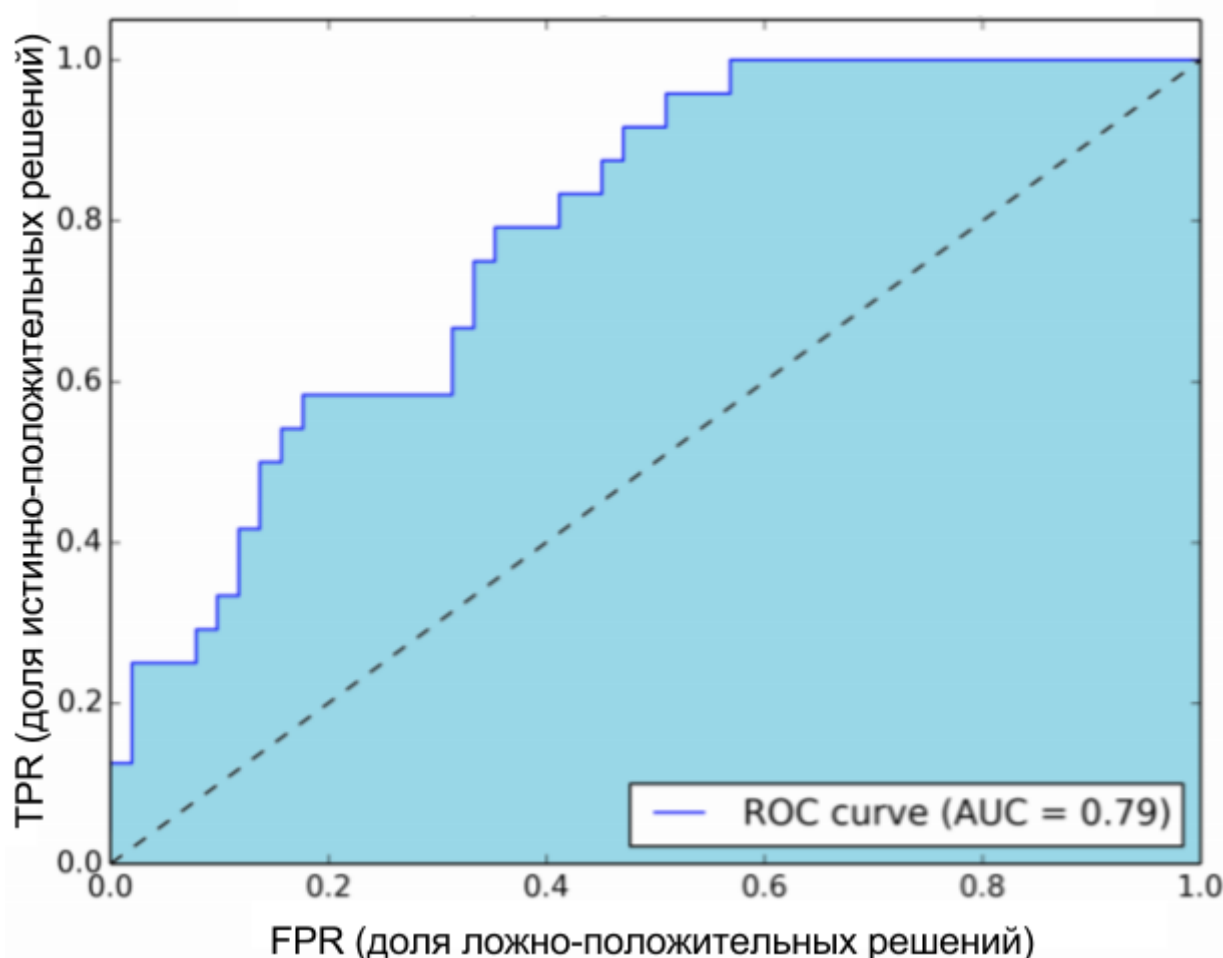


Рис. 3.6. Пример ROC-кривой

Область, закрашенная цветом, и есть искомая характеристика классификатора. Пунктирной линией показана ROC-кривая, соответствующая случайному угадыванию. Таким образом, лучший классификатор будет иметь площадь под ROC-кривой, равную единице, а случайное угадывание соответствует площади 0,5.

Понятно, что классификатор, мало отличающийся от случайного угадывания, вряд ли будет полезен для решения задач.

Лабораторная работа №3. КЛАССИФИКАЦИЯ. ПОСТРОЕНИЕ МОДЕЛИ КЛАССИФИКАЦИИ. СРАВНЕНИЕ РАЗЛИЧНЫХ АЛГОРИТМОВ КЛАССИФИКАЦИИ И ВЫБОР ОПТИМАЛЬНОГО В ORANGE

Цель и задача работы

Изучить основные методы классификации с использованием приложения «Orange Data Mining».

Осуществить классификацию тестовых данных, используя разные алгоритмы. Научиться сравнивать результаты работы алгоритмов классификации и выбирать наиболее подходящий.

Пример построения модели data mining в Orange

Кластеризация с помощью дерева решений

Для загрузки файла используем виджет **File**, который находится на вкладке **Data**. В качестве примера возьмём файл zoo.tab из набора тестовых данных, поставляемых вместе с Orange.

Далее выберем классификатор «Classification Tree» из вкладки **Classify**. Для анализа качества классификации используем виджеты **Test & Score** и **Confusion Matrix**.

На рис. 3.7 показано окно настроек виджета **Test & Score**. В нём можно указать режим работы виджета. Виджет поддерживает различные методы отбора проб (разбиения входных данных на обучающую и тестовую выборки).

1. Cross validation разбивает данные на заданное пользователем количество блоков (обычно 5 или 10). Алгоритм тестируется на примерах из каждого блока, при этом блоки, используемые для обучения и предсказания, постоянно меняются (сначала прогнозируется первый блок, потом второй и так далее, а остальные блоки используются для обучения).

2. Leave-one-out похож на Cross Validation, но он использует в качестве блока только один элемент (т.е. количество блоков будет равно размеру выборки). Этот метод, очевидно, очень стабильный, надежный и очень медленный.

3. Random sampling (случайная выборка) случайным образом разбивает данные на обучающую и тестируемую выборки в указанной пропорции (например, 70:30); вся процедура повторяется в течение определенного количества времени.

4. Test on train data (тест на тренировочных данных) использует весь набор данных для обучения, а затем для тестирования. Этот метод практически всегда дает неправильные результаты.

5. Test on test data (тест на тестовых данных): вышеуказанные методы используют данные только от одного источника данных. Чтобы ввести другой набор данных с примерами тестирования (например, из другого файла или некоторых данных, выбранных в другой виджет), мы выбираем отдельный сигнал проверки данных в канале связи и «Тестирование на тестовых данных».

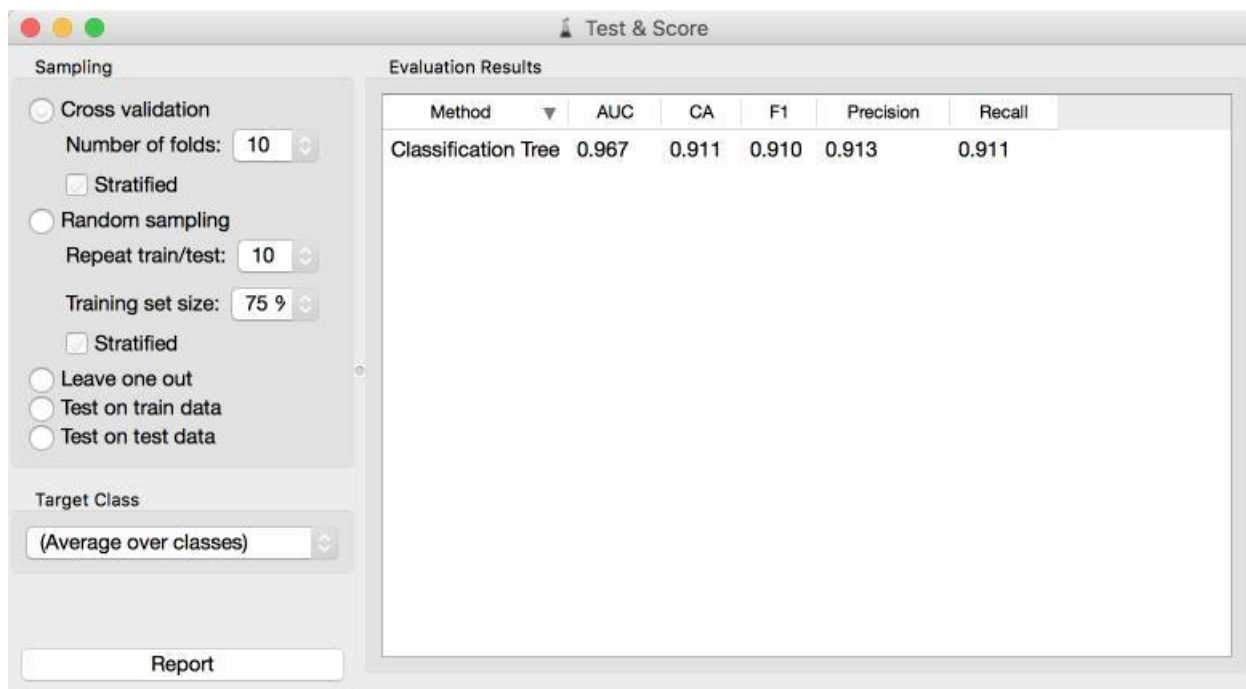


Рис. 3.7. Простой пример классификации с использованием дерева решений

Порядок выполнения работы

1. Получить варианты заданий.
2. Открыть программу Orange.

3. Загрузить тренировочные данные (файл [name].csv) с помощью виджета **File**.

4. Осуществить классификацию данных с помощью алгоритмов Classification Tree, Logistic Regression, Naive Bayes, SVM, CN2 Rule Induction, Nearest Neighbors, Random Forest Classification

5. Осуществить кросс-валидацию с помощью виджета **Test & Score**, используя различные виды разбиения входных данных на тестовые и проверочные (cross validation, random sampling, leave one out).

6. Проверить различные варианты выбора тестовой и обучающей выборки в виджете **Test & Score**.

7. Обосновать выбор наилучшего алгоритма для классификации исходных данных. Использовать виджеты **Confusion Matrix** и **ROC Analysis**.

8. Для алгоритма Classification Tree вывести дерево решений в графическом виде.

9. Вывести ошибки классификации для разных алгоритмов на точечную диаграмму вместе с результатами правильной классификации.

10. Вывести ошибки классификации для разных алгоритмов в виде таблицы.

11. Вывести и проанализировать ROC-кривые для разных алгоритмов.

12. Осуществить классификацию данных файла [name]-not-classified.csv, используя виджет **Predictions**. Вывести полученные результаты в виде таблицы.

13. Обосновать выбор оптимального алгоритма классификации для файла not-classified-xx.csv

14. Подготовить отчёт.

Отчет по работе

1. Титульный лист.

2. Цель работы.

3. Скриншот полной модели data mining.

4. Скриншот окна настроек виджета **Test & Score**.

5. Графическое представление дерева решений.

6. Скриншот совместной точечной диаграммы правильных и ошибочных результатов классификации для каждого используемого алгоритма.
7. Матрицы неточности для самого оптимального и самого неоптимального алгоритмов классификации.
8. Таблицы ошибок классификации для самого оптимального и самого неоптимального алгоритмов классификации.
9. ROC-кривые для всех используемых алгоритмов.
10. Результаты классификации файла [name]-not-classified.csv
11. Выводы.

Контрольные вопросы и задания

1. Что такое классификация? Чем кластеризация отличается от классификации?
2. Ирисы Фишера относительно машинного обучения. Классификатор.
3. Назовите этапы процесса классификации.
4. Опишите проблемы, возникающие при классификации.
5. Опишите алгоритм дерева решений. Приведите простой пример.
6. Опишите наивный байесовский классификатор.
7. Опишите метод опорных векторов.
8. Опишите алгоритм k ближайших соседей.
9. Опишите алгоритм случайного леса.
10. Опишите подходы к определению качества классификации. Способы разделения данных на тестовые и обучающие.
11. Назовите численные показатели качества классификации.
12. Что такое матрица неточности?
13. Опишите F-меру как оценку точности реальных классификаторов.
14. Как анализировать ROC-кривую?
15. Приведите пример применения классификации на реальных данных.

4. РЕКОМЕНДАЦИИ. КОЛЛАБОРАТИВНАЯ ФИЛЬТРАЦИЯ

Выработка рекомендаций в анализе данных представляет собой метод, позволяющий по предпочтениям, оценкам группы людей относительно какого-либо факта, события или явления спрогнозировать предпочтения другой группы и предложить ей соответствующий продукт или услугу. Термин «коллаборативная» означает «совместная». Рекомендательные системы используются, например, на сайте Amazon; в том случае, когда пользователь совершает онлайн покупки на сайте, система отслеживает интересы и предпочтения всех пользователей и «советует», какой товар может заинтересовать пользователя. Также она может, проанализировав предпочтения покупателей по книгам, предложить купить им какой-либо фильм.

Зачастую чтобы получить рекомендацию о каком-либо товаре или услуге, бывает достаточно спросить своих знакомых или людей, мнение которых для вас значимо. Однако данный способ подходит лишь для ограниченного числа альтернатив; в том случае, если количество альтернатив велико, данный метод не будет эффективен в силу отсутствия знаний у респондентов обо всех вариантах. Следовательно, в таких ситуациях целесообразно использование иного метода, в частности, – коллаборативной фильтрации. Термин был предложен в 1992 г.

Д. Голдбергом, разработавшим систему Tapestry для оценки полезности документа пользователями, которая впоследствии использовалась для фильтрации документов другими пользователями. В настоящее время аналогичные методы коллаборативной фильтрации используются на множестве сайтов для рекомендации фильмов, книг, музыки, товаров, услуг и пр. Принцип работы рекомендательных систем заключается в просмотривании большой выборки людей, из которой формируется меньшая выборка с предпочтениями, аналогичными большей. Эта выборка анализируется, мнения и оценки агрегируются, и пользователю выдается рекомендация. Данные системы могут работать с неполными списками оценок или предпочтений и выдавать прогноз, опираясь на эти оценки. Рекомендации для каждого пользователя формируются на основе мнений многих респондентов, при

этом выдаются индивидуально, что отличает метод коллаборативной фильтрации от средней оценки для каждого объекта [14].

Актуальность задачи выработки рекомендаций подтверждается тем, что компания Netflix несколько раз устраивала конкурс по улучшению своей системы рекомендации фильмов. Приз победителю конкурса, улучшившему качество рекомендаций на 10 %, составлял 1 млн долларов!

4.1. Методы построения рекомендательных систем

К методам построения рекомендательных систем относят следующие [15].

1. Content-based. Заключается в построении профилей пользователей и объектов на основе анализа текстовой метаинформации объектов. Затем, с помощью некой меры близости, часто коэффициента Пирсона, определяются объекты, близкие к профилю пользователя. К сожалению, несмотря на богатое наследие Information Retrieval, этот метод пока не удалось заставить работать эффективно. Например, в работе [16] строится система рекомендации интересных твитов в Twitter и явно видно, что вклад в качество системы от content-based компоненты сильно ниже, чем от коллаборативной фильтрации, к которой принадлежат два следующих подхода [15].

2. Neighbourhood-based – подходящими объектами для пользователя считаются объекты, высоко оцениваемые его «соседями», т.е. такими пользователями, у которых оценки хорошо скоррелированы с известными оценками исследуемого пользователя [15].

3. Model-based, matrix factorization (MF) – предполагается, что оценка пользователя определяется не очень большим числом ($K=50-200$) скрытых факторов (например, фильм американский или азиатский; блокбастер или арт-хаус; старый или современный и т.п). Задача состоит в том, чтобы найти такие факторы, при выборе которых эта приближенная модель будет лучше всего приближать реальные пользовательские рейтинги. Для этого часто используется сингулярное разложение матриц [17].

В настоящее время выделяют следующие *подходы к коллаборативной фильтрации*.

1. Методы, базирующиеся на оценках пользователей (memory based), в основе которых статистические методы, которые помогают найти пользователей, мнение которых совпадает с искомым. Таким образом, метод учитывает предшествующие оценки пользователя и оценки других пользователей, имеющих сходные предпочтения с исходным. При выдаче рекомендации по искомому пользователю учитывают меры сходства по полученным данным (корреляция Спирмена, Пирсона и пр.):

- User-based методы основаны на сходстве пользователей и рассчитывают меру близости в режиме онлайн, что сужает область их применения до небольших баз данных. Они имеют высокую точность, но требуют большой ресурсоемкости;

- Item-based методы подразумевают сравнение степени сходства элементов с искомой, которая может осуществляться в отложенном режиме, что позволяет сократить время на создание рекомендаций.

2. Модельные методы (model based) подразумевают анализ модели таким образом, что по оценкам пользователей формируется модель предпочтений пользователей, на основании которой будут выданы рекомендации. В основу этих методов могут быть положены вероятностный подход, кластерный анализ, сингулярное разложение и пр.

3. Гибридные методы включают модельные подходы и методы, связанные с оценками пользователей.

Существуют различные методы построения рекомендаций; далее рассмотрим известный подход к рекомендациям на основе коллаборативной фильтрации – Slope One.

4.2. Алгоритм Slope One

Алгоритм Slope One относится к семейству алгоритмов коллаборативной фильтрации и базируется на отношениях предметов.

Достоинствами данного алгоритма являются возможность снижения эффекта переобучения, увеличение производительности и облегчение интеграции с реальными системами. Для определения оценки пользователя алгоритм использует среднюю разницу в оценках другого пользователя. Slope One проще регрессионного анализа, поскольку он использует регрессию со средней разницей между оценками предметов $f(x)=x+b$ [18].

В табл. 3 представлен принцип работы алгоритма.

Таблица 3

Пример работы алгоритма Slope One

Предмет 1	...	Предмет 2	Пользователь
10	...	15	Анна
...	...	...	...
8	...	?	Александра

Согласно табл. 3 необходимо вычислить оценку Александры по второму предмету. Анна оценила предмет 1 на 10 баллов, а предмет 2 – на 15. В свою очередь, пользователь Александра имеет оценку только для первого предмета. Мы полагаем, что вкусы пользователей похожи, и поэтому будут совпадать и разницы в оценках. Так, разница в оценках Анны по двум предметам равна пяти, причем второй предмет оценен на 5 баллов больше первого, следовательно, Александра оценит предметы с такой же разницей в пользу второго предмета. Значит, ее оценка для второго предмета будет равна 13: $8+(15-10) = 13$.

Рассмотрим более сложный пример с большим числом пользователей и большим количеством отсутствующих оценок, представленный в табл. 4.

Для нахождения оценки пользователя 3 по предмету 3 вычислим среднюю разницу в оценках третьего и первого предметов: $(6+(-3))/2 = 1,5$. Следовательно, оценка третьего предмета выше на 1,5 балла оценки первого предмета. Аналогично вычислим разницы в оценках третьего и второго предметов: $(4+(-2))/2 = 1$.

Таблица 4

Пример для алгоритма Slope One с вычислением средней оценки

Предмет 1	Предмет 2	Предмет 3	Пользователь
6	8	12	1
-	10	8	2
8	5	-	3
10	-	7	4

Таким образом, оценки третьего пользователю по третьему предмету можно спрогнозировать двумя способами: учитывая оценку третьего пользователя для первого предмета: $8 + 1,5 = 9,5$ и учитывая оценку пользователя для второго предмета: $5 + 1 = 6$. Так как пользователь голосовал два раза, у нас есть две возможные оценки, следовательно, итоговую оценку рассчитаем как взвешенное среднее. Для весовых коэффициентов учитываем количество пользователей, давших оценку предмету:

$$\frac{2 * 9,5 + 2 * 6}{4} = 7,75.$$

Подводя итог, отметим, что для использования алгоритма Slope One для n предметов необходимо определить и сохранить среднюю разницу и количество голосов для каждой из n^2 пар предметов.

Недостатком данного подхода является возможный выход за границы допустимых значений (например, используем оценки от 1 до 5, а в результате вычислений для какой-либо оценки получаем среднее значение 8). Поэтому при реализации алгоритма необходимо приводить полученные значения к используемой шкале (в нашем примере, значение 8 будет трактоваться как 5 – максимально возможное).

4.3. Проблемы методов коллаборативной фильтрации

У методов коллаборативной фильтрации есть ряд проблем. Приведем наиболее характерные.

Разреженность данных

Системы выдачи рекомендаций оперируют огромным количеством данных, при этом большинство пользователей не участвуют в оценке товаров и, соответственно, не ставят им оценки. В результате, ранжированная итоговая матрица с оценками пользователей получается громоздкой и содержит много нулевых ячеек, т.е. является разреженной, что может приводить к неэффективности прогнозов, в частности, для новых систем [19]. В некоторых ситуациях разреженность данных может приводить к проблеме нового пользователя (холодного старта).

Проблема нового пользователя

С появлением новых пользователей или объектов оценки возникает проблема подоби́я в силу отсутствия информации. Так, появившиеся пользователи не могут получить отзывов до тех пор, пока сами не оценят те или иные объекты, а последние, в свою очередь, не могут быть рекомендованы, пока не получат оценок. В некоторых случаях для решения этой проблемы может оказаться достаточно перейти к анализу содержимого, поскольку он оперирует признаками предметов. Данное свойство способствует добавлению новых объектов в рекомендации для пользователей. Однако более сложной задачей является проблема получения рекомендации для нового пользователя [19].

Масштабируемость

Следующая проблема связана с ростом количества пользователей в системе. Действительно, при количестве покупателей, равном 10 миллионам $O(M)$, и количестве объектов, равном 1 миллион $O(N)$, метод коллаборативной фильтрации, обладающий сложностью $O(MN)$, становится слишком сложен для расчётов – появляется проблема масштабируемости. Проблема сложности масштабирования также определяется необходимостью мгновенного отклика на онлайн запросы покупателей.

Синонимия

Проблема синонимии, т.е. различного обозначения сходных понятий, присуща большинству рекомендательных систем. В силу сложности языка эти системы имеют трудности при интерпретации слов и трактуют разные слова со сходным значением как разные. Например, «остросюжетный фильм» и «триллер» сходны, но система воспримет их как разные понятия [20].

Мошенничество

Проблема мошенничества, или ложных оценок, остро стоит в рекомендательных системах в первую очередь в связи с коммерческим характером последних. Они активно используются в коммерческих сайтах и могут влиять на продажи, следовательно, заинтересованные пользователи могут ставить высокие оценки своим товарам и снижать рейтинг конкурентов [19].

Разнообразие

Целью методов коллаборативной фильтрации является стремление к открытию и продвижению новых товаров или услуг. В то же время попытка увеличить разнообразие сталкивается со сложностью в продвижении новых продуктов в силу специфики алгоритмов, основанных на продажах и рейтингах. Эти алгоритмы создают сложные условия для продвижения малоизвестных продуктов, так как их замещают популярные продукты, которые давно находятся на рынке. Таким образом, эффект обратно пропорционален и приводит к увеличению однообразия [20].

Белые вороны

При анализе оценок респондентов рекомендательных систем можно выявить пользователей, чье мнение и оценки кардинально отличаются от оценок большинства опрошенных. Эти люди имеют специфический вкус, отличающийся от общепринятого, следовательно, для таких групп пользователей выдача рекомендаций неэффективна. Несмотря на это, сложность заключается не в том, что для таких людей не подходят рекомендательные системы, а в том, что этим пользователям трудно что-либо рекомендовать и в реальной жизни. Поиски путей решения данной задачи ведутся в настоящее время [20].

4.4. Сингулярное разложение матрицы (Matrix Factorization with SVD)

Как было сказано выше, одной из главных проблем коллаборативной фильтрации является проблема разреженности данных. Для решения этой проблемы существует ряд подходов, в частности, методы сингулярного разложения (SVD), главных компонент (PCA), латентно-семантическое индексирование (LSI), основанное на сингулярном разложении. Рассмотрим подробнее основной метод устранения разреженности данных – сингулярное разложение матрицы.

Пусть задана матрица, элементами которой являются оценки пользователей (заданы в строках) относительно товаров (столбцы матрицы) – лайки, факты покупки и пр. Очевидно, что в реальной жизни респондент не оценит все товары, и сами продукты не могут быть оценены большинством пользователей, поэтому матрица разреженная, а также имеет большую размерность.

Выходом из ситуации является сингулярное разложение матрицы R большого размера, но малого ранга f (SVD), определяемое как

$$R = UDV^T, \quad (4.1)$$

где матрица U – размера $n \times n$; V – имеет размер $m \times m$, D – размер $n \times m$ в (4.1). Элементы матрицы D представляют собой сингулярные числа (собственные значения, соответствующие собственным векторам), расположенные по главной диагонали в убывающем порядке, в то время как остальные элементы равны нулю. Следовательно, первые k сингулярных чисел можно оставить, а остальные отбросить, в результате получая наилучшее k -приближение матрицы. Таким образом, метод сингулярного разложения матрицы позволяет получить две матрицы: U – размера $n \times k$, V – размера $m \times k$, поскольку значения сингулярных чисел матрицы D распределяются между матрицами U и V . В нашей задаче n обозначает количество пользователей, m – число объектов, k – число факторов. Факторы определяют товары относительно матрицы V и предпочтения пользователей относительно матрицы U . Это приводит к сильному уменьшению числа параметров. Резюмируя, получим $R_k = U_k D_k V_k^T$, которая будет лучшей аппроксимацией исходной матрицы.

Применение SVD для задач коллаборативной фильтрации подробно описано в [21].

4.5. Оценка качества работы рекомендательных систем

Рассмотрим основные меры качества рекомендательных систем.

Среднеквадратическое отклонение MSE (mean squared error)

В статистике среднеквадратичное отклонение ожидания есть среднее квадратов отклонений прогнозного значения и реального:

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2}.$$

Чем ближе MSE к нулю, тем лучше алгоритм (сумма квадратов ошибок стремится к нулю).

Средняя квадратичная ошибка RMSE (root mean square error)

Средняя квадратичная ошибка – число s , равное квадратному корню из среднего арифметического квадратов данных чисел $a_1, a_2, \dots, a_n$:

$$s = \sqrt{\frac{a_1^2 + a_2^2 + \dots + a_n^2}{n}}.$$

Это значение является мерой несовершенства подгонки оценки к реальным данным.

Погрешность измерений (MAE)

Погрешность измерения – отклонение измеренного значения величины от её истинного (действительного) значения. Погрешность измерения является характеристикой точности измерения. Чем меньше значение MAE, тем лучше алгоритм.

Лабораторная работа №4. РЕКОМЕНДАТЕЛЬНЫЕ СИСТЕМЫ. АЛГОРИТМЫ КОЛЛАБОРАТИВНОЙ ФИЛЬТРАЦИИ. АЛГОРИТМ SLOPE ONE. СРАВНЕНИЕ РЕЗУЛЬТАТОВ РАБОТЫ АЛГОРИТМОВ КОЛЛАБОРАТИВНОЙ ФИЛЬТРАЦИИ В ORANGE

Цель и задача работы

Изучить некоторые алгоритмы коллаборативной фильтрации. Реализовать алгоритм Slope One на языке Python 3. Научиться сравнивать результаты работы алгоритмов коллаборативной фильтрации и выбирать наиболее подходящий для представленных входных данных с использованием приложения «Orange Data Mining».

Ход работы

Использование виджета Python Script в Orange

Виджет **Python Script** [22] позволяет пользователю реализовать любой способ обработки данных, используя язык Python 3. В нашем случае мы должны реализовать различные алгоритмы для вычисления рекомендаций.

Для начала ознакомимся с теми связями (Signals), которые имеет виджет **Python Script**. Signals как раз и являются способом связи виджетов в **Orange**. В Python Script доступны входные переменные `in_data`, `in_distance`, `in_learner`, `in_classifier` и `in_object` (входные данные). Эти переменные могут быть использованы как локальные переменные всего скрипта. Результат работы скрипта можно записать в выходные переменные `out_data`, `out_distance`, `out_learner`, `out_classifier` и `out_object`.

Нам необходимо реализовать всего одно выходное значение – `out_learner`. Для этого мы должны расширить базовый класс `Learner`, доступный в модуле `Orange.base`. Также нам нужен будет класс `Model` из того же модуля `Orange.base`.

Ознакомиться с исходным кодом модуля `Orange.base` можно в [23].

Обязательным условием является использование библиотеки `numpy`.

Во время разработки и тестирования рекомендуется использовать команду `View > Logs` программы Orange, которая позволит выводить произвольные данные в наглядном виде.

Также в процессе разработки рекомендуется сразу подключать реализуемый **Python Script** к виджету **Test & Score** (тип связи out_learner – Learner). Это позволит видеть корректность работы модуля и ошибки в случае их возникновения. Если навести указатель мыши на уведомление об ошибке в любом виджете, то появится более детальное описание возникшей ошибки.

Каждый раз после внесения изменений в **Python Script** необходимо нажать кнопку **Execute**, иначе изменения не вступят в силу. Также можно использовать флаг **Auto Execute**, однако в процессе знакомства и разработки рекомендуется исполнять написанный скрипт явно, так как это сразу покажет результат его выполнения в поле **Console**.

В качестве отправной точки можно выбрать один из алгоритмов классификации, реализованных в Orange, например [24].

Порядок выполнения работы

1. Получить варианты заданий.
2. Ознакомиться с методами оценки качества работы алгоритмов коллаборативной фильтрации.
3. Открыть программу Orange.
4. Загрузить тренировочные данные (файл simple-data.xlsx) с помощью виджета **File**.
5. Реализовать учебный алгоритм, который независимо от входных данных будет выдавать константу. Для этого необходимо использовать виджет **Python Script**. Во время разработки использовать файл «simple-data.xlsx».
6. Реализовать учебный алгоритм, который независимо от входных данных будет выдавать случайный рейтинг в диапазоне от минимального до максимального возможного значения. Для этого использовать виджет **Python Script**. Сравните различные алгоритмы генерации случайных значений и обоснуйте выбор наилучшего. Во время разработки использовать файл «simple-data.xlsx».
7. Реализуйте алгоритм Slope One с помощью виджета **Python Script**. Во время разработки использовать файл «simple-data.xlsx».

8. Осуществить кросс-валидацию нескольких доступных алгоритмов (и обязательно всех реализованных вами) с помощью виджета **Test & Score**, используя различные виды разбиения входных данных на тестовые и проверочные (cross validation, random sampling).

9. Повторить п. 8, используя в качестве входных данных файл data.tab

10. Обосновать выбор наилучшего алгоритма для выработки рекомендаций на основе алгоритмов коллаборативной фильтрации.

11. Подготовить отчёт.

Отчет по работе

1. Титульный лист.

2. Цель работы.

3. Скриншот полной модели data mining.

4. Листинг учебного алгоритма с комментариями внутри кода, всегда возвращающего константу. Обоснование формулы, используемой для вычисления константы.

5. Листинг учебного алгоритма с комментариями внутри кода, возвращающего случайное значение в диапазоне допустимых значений. Обоснование выбора используемого закона распределения случайного значения. Во время разработки использовать файл «simple-data.xlsx».

6. Листинг алгоритма Slope One с комментариями. Во время разработки использовать файл «simple-data.xlsx».

7. Скриншот окна настроек виджета Test & Score для файла «simple-data.xlsx».

8. Скриншот окна настроек виджета Test & Score для файла «data.tab».

9. Выводы о применимости различных алгоритмов для выработки рекомендаций.

10. Выводы.

Контрольные вопросы и задания

1. Что такое коллаборативная фильтрация? Приведите примеры ее использования.

2. Перечислите известные вам алгоритмы коллаборативной фильтрации.
3. Опишите алгоритм Slope One.
4. Сформулируйте проблемы методов коллаборативной фильтрации.
5. Опишите подходы к определению качества рекомендаций, полученных методами коллаборативной фильтрации.
6. Какое значение рейтинга лучше всего выбрать при обработке записи с недостаточным количеством исходных данных? Например, у пользователя нет ни одного известного рейтинга. Объясните свой ответ.
7. Что такое среднеквадратичная ошибка (MSE)?
8. Что такое RMSE?
9. Приведите пример применения коллаборативной фильтрации на реальных данных.

ЗАКЛЮЧЕНИЕ

Настоящее учебное пособие содержит четыре лабораторные работы по курсу «Интеллектуальный анализ данных и биоинспирированные методы и алгоритмы»; в нем рассмотрены основные задачи data mining – кластеризация, анализ текста, классификация, коллаборативная фильтрация, а также методические указания к их выполнению.

Так, разд. 1 освещает задачу кластеризации. В нем приводятся основные определения кластеризации, классификация алгоритмов кластеризации, вводится понятие очистки данных. Лабораторная работа № 1 посвящена задаче кластеризации и построению модели кластеризации в программной среде «Orange». В данной работе магистранты, используя различные алгоритмы кластеризации, изменяя показатели алгоритмов, выявляют оптимальные точки размещения отделений неотложной медицинской помощи в регионе на основе выданных тестовых данных, оценивают влияние фильтрации на конечный результат.

Раздел 2 раскрывает проблему интеллектуального анализа текста, тонального анализа. Вводится понятие Web Mining, приводятся его составляющие. Описываются подходы к решению задачи тонального анализа и проблемы данного направления. Лабораторная работа № 2 посвящена изучению основных методов Web Mining и Text Mining с использованием приложения «Orange Data Mining». В качестве задания предлагается проанализировать реакцию пользователей социальной сети Twitter на различные события, используя разные методы текстового анализа (согласно полученному заданию), а также осуществить поиск закономерностей в текстовых документах.

Задача классификации, ее этапы, различные алгоритмы классификации, способы их сравнения и численные оценки качества алгоритмов, такие как точность, полнота, Confusion Matrix, Roc-кривая, F-мера, приводятся в разд. 3. В качестве примера наглядной классификации рассматриваются ирисы Фишера. Лабораторная работа № 3 посвящена изучению основных методов классификации с использованием приложения «Orange Data Mining». Ее задача – в осуществлении

классификации тестовых данных с использованием различных алгоритмов и сравнении результатов работы алгоритмов классификации, выборе наиболее подходящих, варьируя различные виды разбиения входных данных на тестовые и проверочные.

Раздел 4 рассматривает подходы к организации рекомендательных систем, некоторые алгоритмы коллаборативной фильтрации. В нем раскрывается понятие коллаборативной фильтрации, приводятся примеры ее использования. Освещаются алгоритмы коллаборативной фильтрации, в частности, алгоритм Slope One. Приводятся подходы к определению качества рекомендаций, полученных методами коллаборативной фильтрации. Лабораторная работа № 4 включает задания по реализации алгоритма Slope One на языке Python 3 с последующим сравнением его результатов работы с результатами работы доступных в «Orange» алгоритмов коллаборативной фильтрации и выбором наиболее подходящих для представленных входных данных.

Таким образом, пособие содержит методические указания к выполнению лабораторных работ, а также теоретическую часть, которая помогает магистрантам закрепить теоретический материал и интерпретировать результаты работ, делать обобщенные выводы и прогнозы.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. *Чернышева, Г. Ю.* Интеллектуальный анализ данных [Текст] : учеб. пособие для студентов специальности 080801.65 «Прикладная информатика (в экономике)» / Г. Ю. Чернышева. – Саратов : Саратовский государственный социально-экономический университет, 2012. – 92 с.
2. *Ершов, К. С.* Анализ и классификация алгоритмов кластеризации [Текст] / К. С. Ершов, Т. Н. Романова // Новые информационные технологии в автоматизированных системах. – 2016. – Вып. 19. – С. 274–279.
3. *Чубукова, И. А.* Data Mining [Текст] : учебное пособие / И. А. Чубукова. – Москва : Интернет-Университет Информационных технологий; БИНОМ. Лаборатория знаний, 2006. – 382 с.
4. *Demšar, J.* Orange: Data Mining Fruitful and Fun – A Historical Perspective [Text] / J. Demšar, Z. Blaž // Informatica. – 2013. – Vol. 37. – P. 55–60.
5. *Pang, B.* Opinion Mining and Sentiment Analysis [Text] / B. Pang, L. Lee // Foundations and Trends in Information Retrieval. – 2008. – Vol. 2. – № 1-2. – P. 1–135.
6. Анализ данных и процессов [Текст] / А. Барсегян, М. С. Куприянов, И. И. Холод, М. Д. Тесс, С. И. Елизаров. – 3-е изд. – Санкт-Петербург : БХВ-Петербург, 2009. – 512 с.
7. *Лукашевич, Н. В.* Объектно-ориентированный анализ твитов по тональности: результаты и проблемы [Текст] / Н. В. Лукашевич, Ю. В. Рубцова // Труды Международной конференции DAMDID/RCDL–2015. – Обнинск, 2015. – С. 499–507.
8. *Davidov, D.* Enhanced Sentiment Learning Using Twitter Hashtags and Smileys [Text] / D. Davidov, O. Tsur, A. Rappoport // Proceeding of the 23rd international conference on Computational Linguistics (COLING). – 2010. – P. 241–249.
9. Методы и модели анализа данных: OLAP и Data Mining [Текст] / А. А. Барсегян, М. С. Куприянов, В. В. Степаненко, И. И. Холод. – Санкт-Петербург : БХВ-Петербург, 2004. – 336 с.

10. *Fisher, R. A.* The Use of Multiple Measurements in Taxonomic Problems / R. A. Fisher // *Annals of Eugenics*. – 1936. – Vol. 7. – P. 179–188.
11. *Breiman, L.* Random forests / L. Breiman // *Machine Learning*. – 2001. – Vol. 45(1). – P. 5–32.
12. *Чистяков, С. П.* Случайные леса: обзор / С. П. Чистяков // *Труды Карельского научного центра РАН*. – 2013. – № 1. – С. 117–136.
13. *Соколов, Е.* Семинары по выбору моделей [Электронный ресурс]. – URL: http://www.machinelearning.ru/wiki/images/1/1c/Sem06_metrics.pdf (дата обращения: 30.04.2017).
14. *Сегаран, Т.* Программируем коллективный разум [Текст] : пер. с англ. / Т. Сегаран. – Санкт-Петербург : Символ-Плюс, 2008. – 368 с.
15. *Федоровский, А. Н.* Архитектура рекомендательной системы, работающей на основе неявных пользовательских оценок / А. Н. Федоровский, В. К. Логачева // *Труды 13-й Всероссийской научной конференции «Электронные библиотеки: перспективные методы и технологии, электронные коллекции» – RCDL'2011*. – Воронеж, Россия, 2011. – С. 76–82.
16. *Hannon, J.* Recommending Twitter Users to Follow Using Content and Collaborative Filtering Approaches / J. Hannon, M. Bennett, B. Smyth // In *RecSys '10 Proceedings of the fourth ACM conference on Recommender systems*. Barcelona, Spain, September 26–30, 2010. – P. 199–206.
17. *Koren, Y.* Factorization Meets the Neighborhood: a Multifaceted Collaborative Filtering Model / Y. Koren, P. Ave, F. Park // In: *KDD '08 Proceeding of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining*. – Las Vegas, Nevada, USA, August 24–27, 2008. – P. 426–434.
18. *Lemire, D.* Slope One Predictors for Online Rating-Based Collaborative Filtering [Text] / D. Lemire, A. Maclachlan // *Proceedings of SIAM Data Mining (SDM'05)*. – Newport Beach, California, April 21-23, 2005.
19. *Sammut, C.* Encyclopedia of Machine Learning / C. Sammut, J. Webb. – NY, USA : IBM T. J. Watson Research Center, 2010. – Vol. 1. – P. 829–838. – 1031 p.
20. *Xiaoyuan, S.* A Survey of Collaborative Filtering Techniques / S. Xiaoyuan,

T. M. Khoshgoftaar // Advances in Artificial Intelligence archive, USA. – 2009. – P. 1–19.

21. Латкин, И. И. Коллаборативная фильтрация в рекомендательных системах. SVD-разложение / И. И. Латкин // Электронный журнал «Молодежный научно-технический вестник». – 2015. – № 7.

22. Orange – data mining fruitful and fun [Web resource]. – URL: <http://docs.orange.biolab.si/3/visual-programming/widgets/data/pythonscript.html> (дата обращения: 30.04.2017).

23. Github [Web resource] – URL: <https://github.com/biolab/orange3/blob/master/orange/base.py> (дата обращения: 30.04.2017).

24. GitHub [Web resource]. – URL: https://github.com/biolab/orange3/blob/master/Orange/classification/naive_bayes.py (дата обращения: 30.04.2017).

Учебное издание

Герасименко Евгения Михайловна

**ИНТЕЛЛЕКТУАЛЬНЫЙ АНАЛИЗ ДАННЫХ. АЛГОРИТМЫ DATA
MINING**

Учебное пособие

Редакторы: Проценко И. А., Селезнева Н.И.

Корректоры: Проценко И. А., Селезнева Н.И.

Подписано в печать

Заказ №

Тираж 40 экз.

Формат 60×84^{1/16}. Усл. печ. л. – 5,2.

Уч.-изд. л. – 5,0

Издательство Южного федерального университета

344091, г. Ростов-на-Дону, пр. Стачки, 200/1.

Тел. (863)2478051.

Отпечатано в Отделе полиграфической,
корпоративной и сувенирной продукции

ИПК КИБИ МЕДИА ЦЕНТРА ЮФУ.

ГСП 17 А, Таганрог, 28, Энгельса, 1.

Тел. (8634)371717, 371655.